



ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR

Membre de **HONORIS UNITED UNIVERSITIES**

RAPPORT DE PROJET DE FIN D'ANNÉE (PFA)

2ème année du cycle d'ingénieur — 4IIR

CONCEPTION ET MISE EN ŒUVRE D'UN SOC CLOUD SUR MICROSOFT AZURE POUR LA SUPERVISION DE SECUREVAULT

Wazuh, Suricata, Sysmon, Laravel, Angular et MySQL

Réalisé par : AWFA ABDULKADER	Encadrant professionnel :
Entreprise d'accueil : LTRA TECH	Encadrant pédagogique :
Période du stage :	
Projet de fin d'année — SOC Cloud Azure	

École Marocaine des Sciences de l'Ingénieur — Tanger
Année universitaire 2025–2026

Dédicace

À ma famille, pour sa présence, sa patience et la confiance qu'elle m'accorde dans chaque étape de mon parcours. À toutes les personnes qui m'ont encouragée à poursuivre mes objectifs avec rigueur et persévérance.

Je dédie également ce travail à celles et ceux qui transmettent leurs connaissances et rendent possible l'apprentissage par la pratique.

Remerciements

Je tiens à remercier l'École Marocaine des Sciences de l'Ingénieur pour le cadre pédagogique et les connaissances acquises au cours de ma formation en ingénierie informatique et réseaux. Ces enseignements ont constitué la base nécessaire pour aborder un projet qui combine Cloud, administration Linux, durcissement, détection réseau et supervision de sécurité.

J'adresse également mes remerciements à LTRA TECH pour l'accueil du stage et pour le contexte professionnel dans lequel cette étude a été conduite. La variété des activités numériques de l'entreprise a permis de replacer le sujet SOC dans une problématique concrète de protection des applications, des accès et des données.

Je remercie enfin toutes les personnes qui ont apporté une aide technique, un retour critique ou un soutien durant les phases de configuration, de diagnostic et de validation. Leur disponibilité et leurs conseils ont contribué à l'aboutissement de ce travail.

Résumé

Face à la multiplication des services Cloud, des postes distants et des applications web, la simple mise en place de mécanismes de protection ne suffit plus. Les équipes ont également besoin d'une visibilité continue sur les événements produits par l'infrastructure, le réseau, les systèmes et les applications. Ce projet porte sur la conception et la mise en œuvre d'un centre opérationnel de sécurité, ou SOC, déployé sur Microsoft Azure.

L'architecture réalisée s'appuie sur Wazuh Manager pour l'analyse et la corrélation, Wazuh Indexer pour le stockage, Wazuh Dashboard pour l'investigation, Filebeat pour l'acheminement sécurisé des alertes et Suricata pour la détection réseau. Le serveur Ubuntu hébergeant le socle central a été renforcé à l'aide de règles Network Security Group, d'UFW, d'une configuration SSH restrictive, de Fail2ban, d'options de montage sécurisées et d'audits Ubuntu Security Guide fondés sur le profil CIS Level 1 Server.

Le périmètre a ensuite été élargi à une machine Windows supervisée par un agent Wazuh et Sysmon, puis à SecureVault, une application Laravel, Angular et MySQL destinée au stockage et au partage contrôlé de documents confidentiels. SecureVault produit un journal de sécurité structuré ; l'agent installé sur le serveur web collecte ce journal ainsi que les traces Nginx et les transmet au SOC. Cette diversité de sources permet de rapprocher un événement réseau, une activité endpoint et une action métier.

La validation a été menée maillon par maillon. Un scénario HTTP contrôlé a déclenché la signature Suricata SID 2100498 ; le fichier eve.json a été lu par Wazuh, Filebeat a établi une connexion TLS avec l'indexeur et l'événement est apparu dans Threat Hunting. Les journaux Sysmon de win01 ont également été indexés, notamment un événement de connexion réseau contenant le port 5985 et le contexte du processus. Enfin, les vues et tableaux de bord ont été organisés pour distinguer les événements réseau, Windows et applicatifs.

Le résultat est un prototype SOC complet et démontrable, construit en six phases cohérentes : infrastructure Azure, durcissement, détection réseau, exploitation SIEM, supervision Windows et intégration de SecureVault. Le projet montre surtout qu'une alerte n'a de valeur que si son origine, son transport, son contexte et son interprétation peuvent être vérifiés.

Mots-clés : SOC, Microsoft Azure, Wazuh, Suricata, Filebeat, Sysmon, SIEM, SecureVault, Laravel, Angular, MySQL, CIS Benchmark, Threat Hunting, MITRE ATT&CK.

Abstract

As cloud services, remote endpoints, and web applications expand, preventive controls alone no longer provide sufficient security assurance. Teams also need continuous visibility into events generated by infrastructure, networks, operating systems, and applications. This project focuses on the design and implementation of a Security Operations Center (SOC) hosted on Microsoft Azure.

The implemented architecture relies on Wazuh Manager for analysis and correlation, Wazuh Indexer for storage, Wazuh Dashboard for investigation, Filebeat for secure alert forwarding, and Suricata for network detection. The Ubuntu server hosting the central platform was hardened with Azure Network Security Group rules, UFW, restrictive SSH settings, Fail2ban, secure mount options, and Ubuntu Security Guide audits based on the CIS Level 1 Server profile.

The scope was extended to a Windows endpoint monitored through a Wazuh agent and Sysmon, and then to SecureVault, a Laravel, Angular, and MySQL application designed to store and share confidential documents under controlled conditions. SecureVault writes structured security events, while the web server agent collects application and Nginx logs and forwards them to the SOC. This combination makes it possible to relate network activity, endpoint telemetry, and business actions.

Validation was performed across the full data path. A controlled HTTP scenario triggered Suricata signature SID 2100498; Wazuh read eve.json, Filebeat established a TLS connection to the indexer, and the event became searchable in Threat Hunting. Sysmon events from win01 were also indexed, including a network event containing destination port 5985 and process context. Dedicated views were then organized for network, Windows, and application events.

The final result is a complete and demonstrable SOC prototype delivered through six coherent phases: Azure infrastructure, hardening, network detection, SIEM operations, Windows monitoring, and SecureVault integration. The project demonstrates that an alert is useful only when its origin, transport path, context, and interpretation can all be verified.

Keywords: SOC, Microsoft Azure, Wazuh, Suricata, Filebeat, Sysmon, SIEM, SecureVault, Laravel, Angular, MySQL, CIS Benchmark, Threat Hunting, MITRE ATT&CK.

Table des matières

Introduction générale.....	1
Chapitre 1 : Présentation de LTRA TECH.....	2
1.1 Introduction.....	3
1.2 Fiche d'identité de l'entreprise.....	3
1.3 Mission et positionnement.....	3
1.4 Portefeuille de services.....	3
1.5 Enjeux de cybersécurité associés.....	4
1.6 Conclusion du chapitre.....	5
Chapitre 2 : Contexte et conception du projet.....	6
2.1 Introduction.....	7
2.2 Contexte du stage.....	7
2.3 Contexte et problématique.....	7
2.4 Objectifs et périmètre.....	7
2.5 Besoins fonctionnels et non fonctionnels.....	8
2.6 État de l'art et concepts techniques.....	8
2.7 Choix technologiques.....	9
2.8 Démarche de réalisation.....	9
2.9 Analyse des risques du projet.....	10
2.10 Conclusion du chapitre.....	10
Chapitre 3 : Réalisation et résultats.....	12
3.1 Introduction.....	13
3.2 Environnement de réalisation.....	13
3.3 Architecture déployée du SOC.....	14
3.4 Phase 1 — Infrastructure Azure et socle Wazuh.....	16
3.5 Phase 2 — Durcissement système et réseau.....	19
3.6 Phase 3 — Intégration Suricata/Wazuh.....	27
3.7 Phase 4 — Exploitation SIEM et tableaux de bord.....	36
3.8 Phase 5 — Supervision Windows et corrélation Sysmon.....	39
3.9 Phase 6 — Déploiement et supervision de SecureVault.....	43
3.10 Validation de bout en bout et lecture MITRE ATT&CK.....	49
3.11 Synthèse des résultats.....	50
3.12 Difficultés rencontrées et solutions.....	51
3.13 Compétences mobilisées.....	51
3.14 Conclusion du chapitre.....	52
Conclusion générale.....	53
Bibliographie et webographie.....	54
Annexes techniques.....	55

Liste des figures

Figure 1 : Portefeuille de services de LTRA TECH.....	4
Figure 2 : Position du projet SOC dans l'écosystème numérique étudié.....	5
Figure 3 : Démarche de réalisation du SOC en six phases.....	10
Figure 4 : Caractéristiques de la VM soc01.....	14
Figure 5 : Architecture déployée du SOC Azure.....	15
Figure 6 : Architecture logique du réseau Azure et filtrage NSG.....	16
Figure 7 : Séquence de déploiement de l'infrastructure Azure et du socle Wazuh.....	16
Figure 8 : Service Wazuh Manager actif et sous-processus d'analyse démarrés.....	18
Figure 9 : Diagnostic et résolution de la saturation du disque.....	19
Figure 10 : Défense en profondeur appliquée à soc01.....	20
Figure 11 : Règles entrantes du groupe de sécurité monitoring-nsg (adresses masquées).....	20
Figure 12 : Définition des règles et des politiques par défaut UFW.....	21
Figure 13 : Activation et état du pare-feu UFW sur soc01.....	21
Figure 14 : Authentification SSH par clé et désactivation du mot de passe.....	22
Figure 15 : Contrôles de durcissement SSH issus de l'audit USG.....	22
Figure 16 : Configuration de la jail sshd dans le fichier jail.local.....	23
Figure 17 : Validation de la configuration et de la jail sshd de Fail2ban.....	23
Figure 18 : Service Fail2ban actif sur soc01.....	24
Figure 19 : Réaction de Fail2ban lors du test SSH (adresse masquée).....	24
Figure 20 : Options de montage sécurisées appliquées à /tmp.....	24
Figure 21 : Correction des permissions de journaux et restriction de l'accès SSH à socadmin.....	25
Figure 22 : État initial d'Ubuntu Pro avant l'activation de USG.....	25
Figure 23 : Ré-audit CIS intermédiaire avec six règles en échec.....	26
Figure 24 : Extraction des six règles CIS restant à traiter.....	26
Figure 25 : Évolution du nombre de règles CIS en échec.....	26
Figure 26 : Cinq écarts CIS résiduels après durcissement.....	27
Figure 27 : Chaîne technique de détection et d'indexation Suricata/Wazuh.....	28
Figure 28 : Interface eth0 sélectionnée pour l'acquisition AF_PACKET.....	29
Figure 29 : Plages réseau déclarées dans HOME_NET.....	29
Figure 30 : Échec initial du test Suricata en l'absence de fichier de règles.....	29
Figure 31 : Téléchargement et activation des règles Emerging Threats.....	30
Figure 32 : Chargement des règles et initialisation des journaux Suricata.....	30
Figure 33 : Présence de la signature SID 2100498 dans le jeu de règles.....	31
Figure 34 : Service Suricata actif sur l'interface eth0.....	31
Figure 35 : Production d'événements structurés dans eve.json.....	31
Figure 36 : Déclaration de eve.json dans la configuration Wazuh.....	32
Figure 37 : Lecture de eve.json confirmée par Wazuh Logcollector.....	33
Figure 38 : Requête HTTP contrôlée utilisée pour le test IDS.....	33
Figure 39 : Déclenchement de la signature Suricata SID 2100498.....	34
Figure 40 : Service Filebeat actif après la configuration de la chaîne Wazuh.....	34

Figure 41 : Test de sortie Filebeat vers le Wazuh Indexer avec TLS.....	34
Figure 42 : Événement du groupe Suricata visible dans Wazuh Threat Hunting.....	35
Figure 43 : Architecture finale du laboratoire SOC sur Microsoft Azure.....	36
Figure 44 : Démarche de validation de la Phase 4 dans Wazuh.....	37
Figure 45 : Construction d'une recherche Threat Hunting appliquée aux événements Suricata.....	38
Figure 46 : Organisation du tableau de bord de supervision des événements de sécurité.....	39
Figure 47 : Intégration de la machine Windows win01 au réseau Azure.....	40
Figure 48 : Chaîne d'enregistrement et de supervision de l'agent Windows.....	40
Figure 49 : Sources Windows et canal Microsoft-Windows-Sysmon/Operational.....	41
Figure 50 : Champs observés dans un événement réseau Sysmon indexé.....	42
Figure 51 : Organisation du tableau de bord Windows Monitoring.....	43
Figure 52 : Périmètre fonctionnel et données sensibles de SecureVault.....	43
Figure 53 : Topologie de déploiement de web01 et filtrage réseau associé.....	44
Figure 54 : Principaux modules fonctionnels de l'application SecureVault.....	45
Figure 55 : Architecture applicative Nginx, Angular, Laravel et MySQL.....	45
Figure 56 : Structure d'un événement de sécurité écrit dans security.log.....	46
Figure 57 : Sources SecureVault et Nginx déclarées dans l'agent Wazuh.....	47
Figure 58 : Flux autorisés entre web01 et soc01.....	48
Figure 59 : Investigation d'un événement SecureVault dans Wazuh.....	49
Figure 60 : Chemin complet d'un événement, de SecureVault au Dashboard.....	50
Figure 61 : Répartition des capacités de détection validées par source.....	51

Liste des tableaux

Tableau 1 : Fiche d'identité de LTRA TECH.....	3
Tableau 2 : Services de LTRA TECH et enjeux de cybersécurité.....	4
Tableau 3 : Objectifs opérationnels du projet.....	7
Tableau 4 : Besoins fonctionnels.....	8
Tableau 5 : Besoins non fonctionnels.....	8
Tableau 6 : Comparaison des rôles de Wazuh et Suricata.....	9
Tableau 7 : Choix technologiques retenus.....	9
Tableau 8 : Démarche de réalisation du SOC.....	10
Tableau 9 : Caractéristiques de l'environnement Azure.....	13
Tableau 10 : Rôle des composants de l'architecture déployée.....	15
Tableau 11 : Chronologie de l'incident de capacité disque.....	19
Tableau 12 : Écarts CIS résiduels.....	27
Tableau 13 : Contrôles de durcissement appliqués.....	27
Tableau 14 : Paramètres de fonctionnement de Suricata.....	28
Tableau 15 : Problèmes, analyses et solutions de la Phase 3.....	35
Tableau 16 : Synthèse des résultats techniques.....	1
Tableau 17 : Matrice synthétique des résultats techniques.....	56
Tableau 18 : Inventaire final des machines et rôles de l'architecture.....	36
Tableau 19 : Requêtes et vues utilisées pendant l'investigation SIEM.....	38
Tableau 20 : Sources de journaux collectées sur win01.....	41
Tableau 21 : Champs utiles d'un événement réseau Sysmon.....	42
Tableau 22 : Fonctions SecureVault et événements de sécurité associés.....	44
Tableau 23 : Composants de la pile applicative déployée sur web01.....	45
Tableau 24 : Structure d'un événement applicatif SecureVault.....	46
Tableau 25 : Règles Wazuh adaptées aux événements SecureVault.....	48
Tableau 26 : Correspondance entre scénarios observables et MITRE ATT&CK.....	50
Tableau 27 : Résultats finaux des six phases.....	50
Tableau 28 : Difficultés, causes et solutions mises en œuvre.....	51

Liste des abréviations

Abréviation	Signification
AF_PACKET	Interface Linux de capture de paquets utilisée par Suricata
API	Application Programming Interface
CIS	Center for Internet Security
DMZ	Demilitarized Zone
EVE JSON	Extensible Event Format JSON de Suricata
FIM	File Integrity Monitoring
IDS / NIDS	Intrusion Detection System / Network Intrusion Detection System
JSON	JavaScript Object Notation
KQL	Kibana Query Language, syntaxe de filtrage utilisée dans les vues
NSG	Network Security Group
RDP	Remote Desktop Protocol
RFC1918	Plages privées IPv4 définies par la RFC 1918
SIEM	Security Information and Event Management
SID	Signature Identifier d'une règle Suricata
SOC	Security Operations Center
SSH	Secure Shell
Sysmon	System Monitor de Microsoft Sysinternals
TLS	Transport Layer Security
USG	Ubuntu Security Guide
VM	Virtual Machine
VNet	Azure Virtual Network

Introduction générale

La transformation numérique a déplacé une partie importante des activités vers des environnements Cloud et des applications accessibles à distance. Cette évolution facilite le travail et le partage de l'information, mais elle augmente aussi le nombre de points à surveiller : interfaces web, comptes utilisateurs, machines virtuelles, services d'administration et flux entre systèmes. Une configuration sécurisée réduit le risque, sans pour autant expliquer ce qui se passe lorsqu'un événement suspect se produit.

Le SOC répond à ce besoin de visibilité. Il centralise des journaux issus de sources différentes, les analyse, fait ressortir les événements significatifs et fournit à l'analyste les éléments nécessaires à l'investigation. Sa valeur ne dépend donc pas uniquement du nombre d'outils installés. Elle repose sur la continuité de la chaîne de données, depuis la génération d'une trace jusqu'à sa recherche dans le SIEM, ainsi que sur la capacité à replacer cette trace dans son contexte.

Le présent projet consiste à concevoir et déployer sur Microsoft Azure un SOC destiné à superviser un serveur Linux, un poste Windows, le trafic réseau et une application web métier. L'application retenue, SecureVault, est un coffre documentaire privé développé avec Laravel, Angular et MySQL. Elle permet de gérer des documents sensibles, des dossiers, des liens temporaires de partage, des rôles et un historique d'audit. Ces fonctions donnent au laboratoire un actif réaliste : une compromission pourrait exposer des documents, des comptes et des preuves d'activité.

La problématique peut ainsi être formulée de la manière suivante : comment construire une architecture SOC Cloud sécurisée, capable de centraliser des journaux Linux, réseau, Windows et applicatifs, puis de transformer ces données hétérogènes en informations exploitables pour la détection et l'investigation ?

La réponse proposée associe Microsoft Azure, Ubuntu, Wazuh, Filebeat, Suricata, Sysmon et une instrumentation spécifique de SecureVault. La réalisation commence par le réseau et les services centraux, puis renforce l'hôte avant d'ajouter les sources de détection. Cette progression permet de tester chaque dépendance séparément et d'éviter qu'une erreur de collecte soit confondue avec une erreur de détection ou d'indexation.

Le rapport est organisé en trois chapitres. Le premier présente le cadre du stage. Le deuxième expose la problématique, les besoins, l'architecture et les choix technologiques. Le troisième décrit les six phases de réalisation, les commandes essentielles, les incidents rencontrés, les preuves obtenues et la lecture des résultats à l'aide de MITRE ATT&CK. Les tests offensifs non nécessaires à la validation de l'intégration applicative ont été exclus du périmètre final ; l'analyse repose sur des événements contrôlés et sur les journaux effectivement collectés.

Chapitre 1

Présentation de LTRA TECH

1.1 Introduction

LTRA TECH est une entreprise marocaine spécialisée dans les solutions digitales et le développement technologique. Basée à Tanger, elle accompagne les organisations dans leur transformation numérique en leur proposant des solutions adaptées à leurs besoins et à leurs objectifs métiers. Son offre couvre notamment la création de sites web et d'applications, l'identité visuelle, le marketing numérique, l'intelligence artificielle, la data, l'automatisation, la maintenance applicative et le Cloud computing [1][2].

La diversité de ces activités implique la manipulation de comptes utilisateurs, de données métiers, d'applications et d'infrastructures numériques. La confidentialité, l'intégrité et la disponibilité de ces ressources constituent donc des enjeux importants pour la continuité des services proposés.

Ce chapitre présente l'identité de LTRA TECH, sa mission, son positionnement et ses principales activités, avant de montrer le lien entre son environnement technologique et le projet de conception d'un SOC Cloud sur Microsoft Azure.

1.2 Fiche d'identité de l'entreprise

Tableau 1 : Fiche d'identité de LTRA TECH

Élément	Information
Dénomination	LTRA TECH
Secteur d'activité	Solutions digitales et développement technologique
Localisation	9 Rue Uruguay, Résidence Chaïmae, Tanger, Maroc
Téléphone	+212 752 744 001
Site web	www.ltratech.com
Marché	Accompagnement des entreprises dans leur transformation digitale

1.3 Mission et positionnement

LTRA TECH a pour mission d'accompagner les entreprises dans leur transformation digitale en concevant des solutions technologiques adaptées à leurs besoins et à leurs objectifs métiers [1]. Son approche repose sur l'analyse du besoin, le choix des technologies appropriées et la mise en place de solutions capables d'améliorer l'efficacité, la visibilité et la qualité des services numériques.

L'entreprise se positionne comme un partenaire de transformation digitale et d'innovation technologique au Maroc. Elle intervient dans des domaines complémentaires qui couvrent la conception, le développement, l'exploitation et l'évolution des solutions numériques.

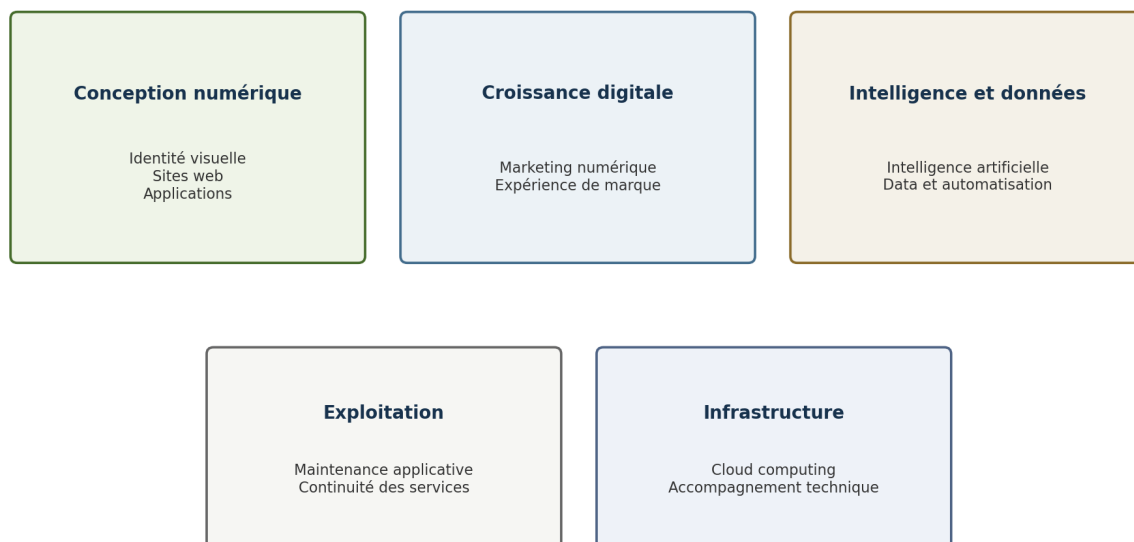
Ce positionnement place LTRA TECH au cœur d'enjeux liés à la disponibilité des services, à la protection des données et à la sécurité des infrastructures. Le projet SOC s'inscrit dans cette orientation en apportant une capacité centralisée de surveillance, de détection et d'analyse des événements de sécurité.

1.4 Portefeuille de services

LTRA TECH propose un portefeuille de services couvrant plusieurs domaines de la transformation numérique. L'entreprise intervient notamment dans la création d'identités visuelles, le développement de sites web et d'applications, le marketing numérique, l'intelligence artificielle, la data, l'automatisation, la maintenance applicative et le Cloud computing [1].

Cette complémentarité permet d’accompagner les clients depuis la conception de leur solution jusqu’à son hébergement, sa maintenance et son évolution. Elle met également en évidence la nécessité de protéger les comptes, les données, les applications et les environnements techniques mobilisés.

Offre de services publiée par LTRA TECH



Source : site officiel de LTRA TECH, consulté le 21 août 2026.

Figure 1 : Portefeuille de services de LTRA TECH

La Figure 1 regroupe les principales familles de services de LTRA TECH. Elle illustre la continuité entre la conception, le développement, la valorisation des données et l’exploitation des services numériques.

Tableau 2 : Services de LTRA TECH et enjeux de cybersécurité

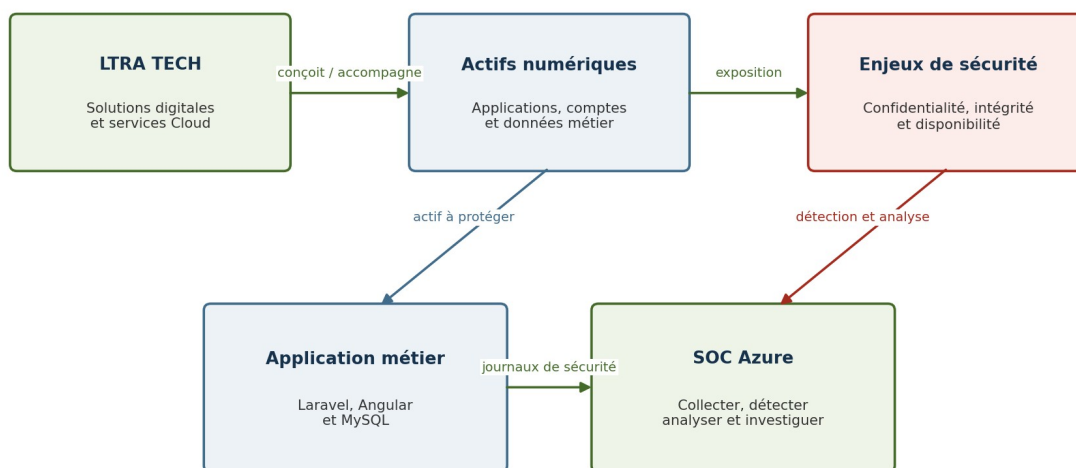
Famille	Services	Enjeux associés
Conception	Identité visuelle et image de marque	Intégrité des contenus et maîtrise des accès
Développement	Sites web et applications	Protection des comptes, des données et des interfaces
Croissance	Marketing numérique	Sécurisation des comptes et suivi des changements
Intelligence	IA, data et automatisation	Traçabilité des traitements et protection des données
Exploitation	Maintenance applicative	Disponibilité, correctifs et gestion des incidents
Infrastructure	Cloud computing	Contrôle des accès, filtrage réseau et supervision

1.5 Enjeux de cybersécurité associés

Les services numériques s’appuient sur des comptes, des serveurs, des applications et des bases de données exposés à plusieurs catégories de menaces : accès non autorisés, erreurs de configuration, vol d’identifiants, exploitation de vulnérabilités, interruption de service et altération des données. Une protection efficace doit donc combiner des mesures préventives et une surveillance continue.

Le SOC fournit cette visibilité en centralisant les journaux techniques et en faisant ressortir les événements nécessitant une investigation. Dans le cadre du projet, Microsoft Azure fournit l’infrastructure Cloud, Wazuh assure la centralisation et l’analyse, tandis que Suricata apporte la détection réseau.

Position du projet SOC dans l'écosystème numérique



Le SOC relie les événements techniques aux actifs numériques et à leurs enjeux métier.

Figure 2 : Position du projet SOC dans l'écosystème numérique étudié

La Figure 2 positionne le SOC entre les actifs numériques et les risques auxquels ils sont exposés. L'objectif est de relier un événement technique à la ressource concernée et à son impact potentiel sur l'activité.

1.6 Conclusion du chapitre

Ce chapitre a présenté LTRA TECH, son positionnement et ses principaux domaines d'activité. Grâce à un portefeuille couvrant le développement web, les applications, le Cloud, la data, l'intelligence artificielle, l'automatisation et la maintenance, l'entreprise accompagne ses clients dans la réalisation et l'évolution de leurs projets numériques.

Ces activités reposent sur des infrastructures et des données qui doivent être protégées contre les cybermenaces. Le projet de mise en œuvre d'un SOC Cloud s'inscrit ainsi dans un contexte cohérent avec les enjeux technologiques de LTRA TECH. Le chapitre suivant présente le contexte, la problématique, les objectifs et les choix techniques du projet.

Chapitre 2

Contexte et conception du projet

2.1 Introduction

Ce chapitre traduit le besoin de supervision en exigences techniques. Il présente le périmètre à protéger, les fonctions attendues du SOC, les choix d'architecture et la démarche qui relie les six phases de réalisation.

2.2 Contexte du stage

Le projet se situe à l'intersection du Cloud, de l'administration Linux et Windows, de la détection réseau, du développement web et de l'analyse de journaux. L'environnement comporte trois rôles principaux : soc01 pour les composants centraux de supervision, win01 pour la télémétrie endpoint et web01 pour l'application SecureVault.

SecureVault constitue l'actif métier. La plateforme stocke des documents privés, contrôle leur partage et conserve l'historique des actions sensibles. Le SOC doit donc être capable d'observer non seulement l'état des serveurs, mais aussi les connexions, les processus Windows, les requêtes web et les actions métier enregistrées par l'application.

2.3 Contexte et problématique

Les sources de sécurité n'utilisent pas le même format ni le même niveau de détail. Suricata décrit le trafic dans EVE JSON ; Windows produit des événements EventChannel enrichis par Sysmon ; Nginx journalise les requêtes HTTP ; Laravel écrit les erreurs et les événements métier. Sans normalisation et sans centralisation, une action peut apparaître comme plusieurs traces isolées et perdre sa signification.

La problématique retenue est donc la suivante : comment déployer sur Azure une chaîne de supervision sécurisée et lisible, capable de collecter ces sources hétérogènes, de les indexer et de fournir une investigation cohérente autour de SecureVault ?

2.4 Objectifs et périmètre

Tableau 3 : Objectifs opérationnels du projet

Objectif	Mise en œuvre	Critère de réussite
Construire le socle Cloud	VNet, sous-réseaux, NSG et VM soc01	Services centraux accessibles et contrôlés
Réduire la surface d'attaque	SSH, UFW, Fail2ban, permissions et CIS	Paramètres appliqués et audités
Détecter le trafic	Suricata, règles Emerging Threats et EVE JSON	Signature de test détectée
Exploiter le SIEM	Filebeat, Indexer, Threat Hunting et dashboards	Événements recherchables et vues sauvegardées
Superviser Windows	Agent Wazuh et Sysmon sur win01	Événements endpoint indexés
Superviser SecureVault	Agent web01, logs Nginx/Laravel/security.log	Actions applicatives centralisées

Le périmètre couvre le prototype Azure et les événements produits dans ce laboratoire. Il n'inclut pas un centre de surveillance 24 h/24, un cluster Wazuh à haute disponibilité, une certification de conformité ni une campagne offensive exhaustive. Ces éléments relèvent d'une industrialisation, alors que le présent travail évalue la cohérence technique et la traçabilité du prototype.

2.5 Besoins fonctionnels et non fonctionnels

2.5.1 Besoins fonctionnels

Tableau 4 : Besoins fonctionnels

ID	Besoin	Mécanisme
BF-01	Contrôler les flux entre Internet et les VM	NSG Azure et pare-feu local
BF-02	Administrer les serveurs de manière sécurisée	SSH par clé et RDP restreint
BF-03	Détecter une activité réseau	Suricata et règles de signature
BF-04	Collecter les événements Linux et Windows	Wazuh Logcollector et agents
BF-05	Acheminer et indexer les alertes	Filebeat avec TLS et Wazuh Indexer
BF-06	Rechercher et visualiser les événements	Threat Hunting, Discover et dashboards
BF-07	Tracer les actions sensibles de SecureVault	security.log structuré et audit applicatif
BF-08	Relier les événements à des scénarios d'attaque	Règles Wazuh et MITRE ATT&CK

2.5.2 Besoins non fonctionnels

Tableau 5 : Besoins non fonctionnels

Exigence	Application	Contrôle
Confidentialité	Protéger comptes, documents et secrets	Filtrage, droits et masquage des preuves
Intégrité	Préserver configurations et journaux	Permissions, FIM et validation de syntaxe
Disponibilité	Maintenir la chaîne de supervision	systemd, contrôle disque et services
Traçabilité	Relier une action à son auteur et son contexte	horodatage, IP, utilisateur et request_id
Maintenabilité	Isoler rapidement une rupture de flux	tests indépendants de chaque maillon
Évolutivité	Ajouter de nouvelles sources	agents, localfile JSON et règles personnalisées

2.6 État de l'art et concepts techniques

2.6.1 SOC, SIEM et XDR

Un SIEM centralise et rend interrogeables des événements provenant de plusieurs composants. Wazuh associe à cette fonction des capacités endpoint, d'analyse de configuration et d'intégrité. Le Manager traite les événements ; Filebeat les transmet à l'Indexer avec TLS ; le Dashboard interroge les index et fournit les vues d'investigation [5].

2.6.2 Détection réseau avec Suricata

Suricata inspecte les paquets et applique un jeu de règles. Son format EVE JSON conserve des champs structurés qui peuvent être décodés et filtrés dans Wazuh. La configuration dépend de l'interface d'écoute, de HOME_NET, du jeu de signatures et de la validité du fichier suricata.yaml [6][8].

2.6.3 Télémétrie Windows avec Sysmon

Sysmon complète les journaux Windows en enregistrant notamment les créations de processus, les connexions réseau et certaines opérations sur les fichiers. Il ne décide pas qu'une action est malveillante : il produit un contexte que le SIEM doit analyser [17].

2.6.4 Journalisation applicative

Laravel permet d'écrire des messages dans des canaux dédiés et d'ajouter un contexte structuré. Dans SecureVault, cette possibilité est utilisée pour séparer les événements de sécurité des erreurs générales. Wazuh peut lire un fichier JSON grâce au bloc localfile, puis appliquer des règles adaptées au vocabulaire de l'application [7][20].

2.6.5 Durcissement et CIS Benchmark

Ubuntu Security Guide automatise l'audit du système par rapport à des profils CIS. Le score sert de repère pour prioriser les corrections ; il ne remplace pas l'analyse de contexte et ne constitue pas, à lui seul, une certification [9][10].

Tableau 6 : Comparaison des rôles de Wazuh et Suricata

Source	Donnée principale	Point d'exploitation
Suricata	EVE JSON, signatures et métadonnées réseau	Wazuh Threat Hunting
Ubuntu / Wazuh	journaux système, SCA et alertes	Indexer et Dashboard
Windows / Sysmon	EventChannel, processus et connexions	Discover et Windows Monitoring
Nginx	requêtes, codes HTTP et erreurs	analyse des accès web
SecureVault	authentification, partage, rôles et audit	règles et vue applicative

2.7 Choix technologiques

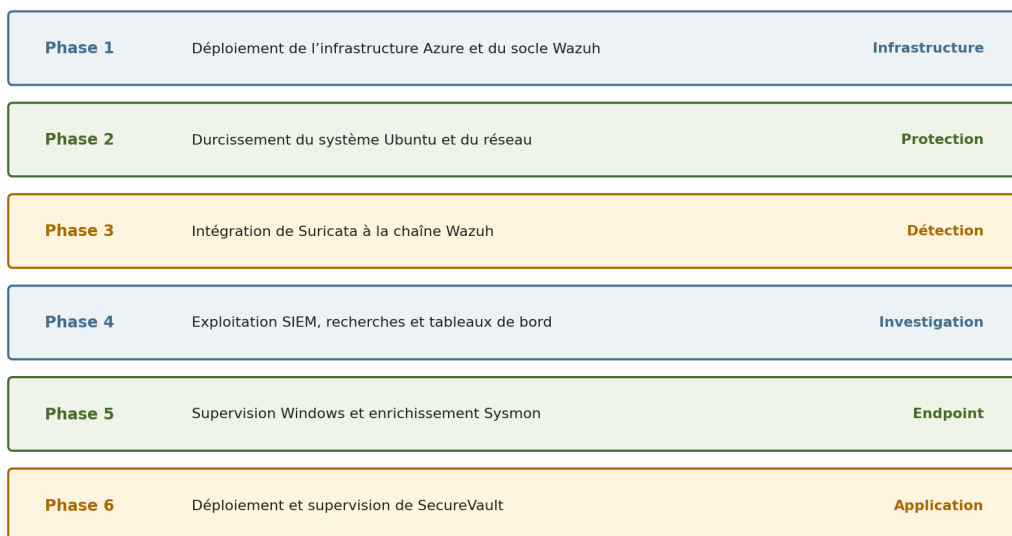
Tableau 7 : Choix technologiques retenus

Technologie	Rôle	Justification
Microsoft Azure	Hébergement, VNet, sous-réseaux et NSG	Infrastructure reproductible et segmentation réseau
Ubuntu Server	Système de soc01 et web01	Administration sans interface graphique et support LTS
Wazuh	Collecte, analyse, indexation et visualisation	Pile SIEM/XDR open source intégrée
Suricata 7.0.3	Détection réseau	Signatures et événements EVE JSON
Filebeat	Transport des alertes	Connexion TLS vers Wazuh Indexer
Windows + Sysmon	Source endpoint	Contexte détaillé sur processus et connexions
Nginx / PHP-FPM	Publication de SecureVault	Serveur web et exécution Laravel
Laravel / Angular / MySQL	Application métier	API, interface, données et audit

2.8 Démarche de réalisation

1. Déployer le réseau Azure, soc01 et les composants centraux de Wazuh.
2. Durcir Ubuntu et les accès avant d'élargir l'exposition du laboratoire.
3. Configurer Suricata et suivre un événement réseau jusqu'au SIEM.
4. Structurer les recherches Threat Hunting et les tableaux de bord.
5. Ajouter win01, l'agent Wazuh et la collecte Sysmon.
6. Déployer SecureVault sur web01 et centraliser ses journaux.

Démarche de réalisation du SOC



La progression va de l'infrastructure à l'exploitation, puis à la supervision des postes et de l'application.

Figure 3 : Démarche de réalisation du SOC en six phases

Tableau 8 : Démarche de réalisation du SOC

Phase	Finalité	Travaux	Résultat
1	Infrastructure	Azure, réseau, VM et Wazuh	Socle central disponible
2	Protection	NSG, UFW, SSH, Fail2ban et CIS	Surface d'attaque réduite
3	Détection	Suricata, EVE JSON et collecte	Alerte réseau centralisée
4	Investigation	requêtes, labels et dashboards	Vues SOC exploitables
5	Endpoint	agent Windows et Sysmon	Télémétrie win01 indexée
6	Application	web01, SecureVault et logs	Événements métier supervisés

2.9 Analyse des risques du projet

Le SOC est lui-même un actif sensible. Une perte d'accès, une saturation du stockage ou une règle réseau trop large peut interrompre la supervision ou exposer des données. Les changements ont donc été appliqués avec une vérification avant/après et les captures ont été expurgées des secrets.

- Accès administratif : limiter SSH, RDP et Dashboard aux sources nécessaires.
- Capacité : surveiller la partition racine, les index et les répertoires temporaires.
- Collecte : vérifier séparément la production du log, sa lecture, son transport et son indexation.
- Application : ne jamais écrire de mot de passe, jeton ou contenu documentaire dans security.log.
- Documentation : masquer les IP publiques, clés, cookies et identifiants sensibles.

2.10 Conclusion du chapitre

Le projet a été conçu comme une chaîne multi-source plutôt que comme une addition d'outils. Chaque phase ajoute une capacité vérifiable au socle précédent : protéger, détecter, collecter, indexer, investiguer et

relier les événements à un actif métier. Le chapitre suivant présente la réalisation et les résultats obtenus sur les six phases.

Chapitre 3

Réalisation et résultats

3.1 Introduction

La réalisation a été conduite en six phases successives. Les trois premières établissent et sécurisent le socle de supervision ; les trois suivantes transforment ce socle en environnement SOC multi-source, avec des vues d'investigation, une machine Windows et l'application SecureVault. L'ordre retenu est important : il évite d'ajouter une source tant que la chaîne centrale n'est pas stable.

Les sections suivantes décrivent les commandes utiles, mais surtout la raison de chaque choix et la preuve recherchée. Une commande réussie n'est pas considérée comme un résultat suffisant si le service attendu, le journal ou l'événement final ne peut pas être observé.

3.2 Environnement de réalisation

L'environnement est hébergé dans Microsoft Azure et utilise soc-vnet, dont le préfixe est 10.0.0.0/16. soc01, sous Ubuntu Server 24.04.4 LTS, héberge les composants centraux Wazuh et Suricata. win01 est une machine Windows ajoutée au même réseau virtuel ; son adresse privée observée est 10.0.0.4. web01 héberge SecureVault sur Ubuntu, sans interface graphique.

Tableau 9 : Caractéristiques de l'environnement Azure

Élément	Configuration	Fonction
Groupe de ressources	azure-soc-rg - France Central	Regroupement et cycle de vie du laboratoire
Réseau virtuel	soc-vnet - 10.0.0.0/16	Communication privée entre les VM
soc01	Ubuntu 24.04.4 LTS - 10.0.2.4	Wazuh, Filebeat, Indexer, Dashboard et Suricata
win01	Windows - 10.0.0.4	Agent Wazuh, journaux Windows et Sysmon
web01	Ubuntu Server	Nginx, Angular, Laravel, MySQL et agent Wazuh
Poste Kali	Hôte de test autorisé	Trafic contrôlé et vérifications réseau

```

MINGW64/c/Users/Awfa
AwFa@Legion MINGW64 ~
$ ssh -i "C:\Users\Awfa\Documents\pfa Azure\soc01_key.pem" socadm
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-1022-azure x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

System information as of Fri Aug 14 11:51:33 UTC 2026

System load: 0.25          Processes:            139
Usage of /:  99.9% of 28.02GB Users logged in:        0
Memory usage: 28%          IPv4 address for eth0: 10.0.2.4
Swap usage:  0%

=> / is using 99.9% of 28.02GB

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
   just raised the bar for easy, resilient and secure K8s cluster deployment.

   https://ubuntu.com/engage/secure-kubernetes-at-the-edge

Expanded Security Maintenance for Applications is not enabled.

13 updates can be applied immediately.
10 of these updates are standard security updates.
To see these additional updates run: apt list --upgradable

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

Last login: Tue Aug 11 14:04:59 2026 from [redacted]
socadmin@soc01:~$ sudo pro status
SERVICE AVAILABLE DESCRIPTION
anbox-cloud yes Scalable Android in the cloud
esm-apps yes Expanded Security Maintenance for Applications
esm-infra yes Expanded Security Maintenance for Infrastructure
fips-updates yes FIPS compliant crypto packages with stable security updates
landscape yes Management and administration tool for Ubuntu
livepatch yes Canonical Livepatch service
realtime-kernel yes Ubuntu kernel with PREEMPT_RT patches integrated
usg yes Security compliance and audit tools

For a list of all Ubuntu Pro services, run 'pro status --all'

This machine is not attached to an Ubuntu Pro subscription.
See https://ubuntu.com/pro
socadmin@soc01:~$ sudo pro enable usg
Cannot enable unknown service 'usg'.

socadmin@soc01:~$ |

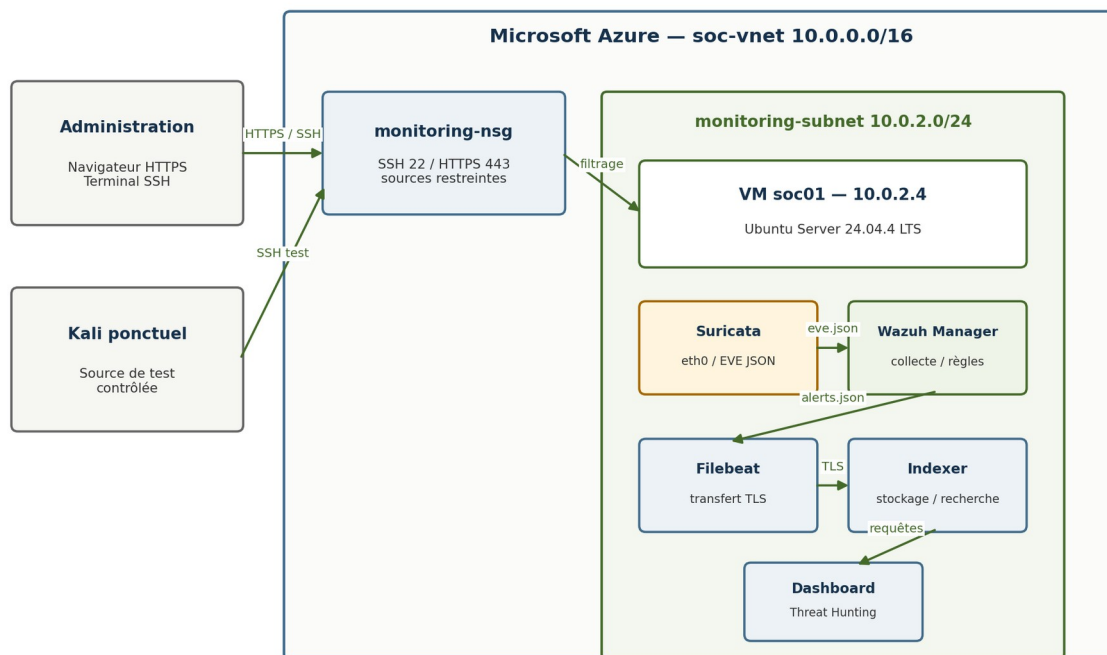
```

Figure 4 : Caractéristiques de la VM soc01

3.3 Architecture déployée du SOC

Le cœur de la plateforme est déployé en mode all-in-one sur soc01. Les rôles restent logiquement séparés : Suricata produit les événements réseau ; Wazuh Manager les analyse ; Filebeat transfère les alertes ; l'Indexer les stocke ; le Dashboard les rend consultables. Les agents de win01 et web01 ajoutent ensuite leurs journaux par le réseau privé Azure.

Architecture déployée du SOC Azure



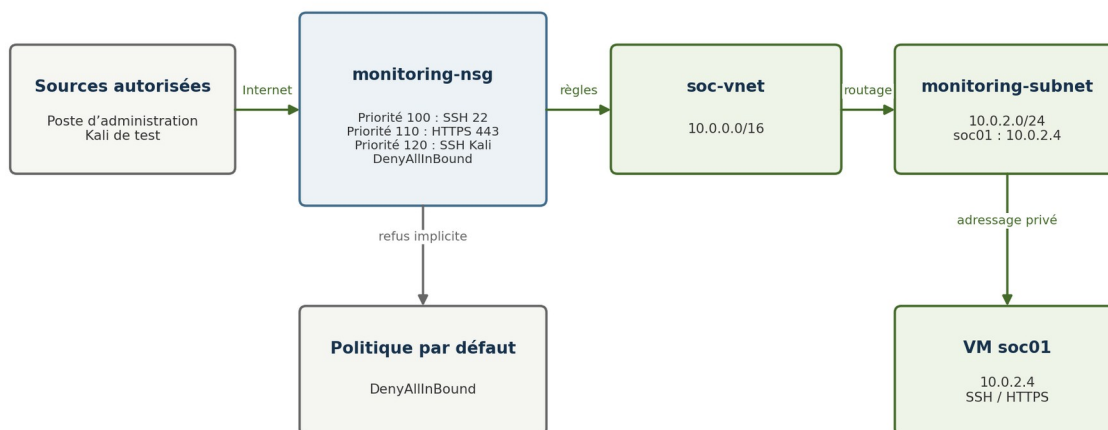
Les composants de détection, d'analyse, de transport, d'indexation et de visualisation sont regroupés sur soc01.

Figure 5 : Architecture déployée du SOC Azure

Tableau 10 : Rôle des composants de l'architecture déployée

Composant	Fonction	Donnée
Suricata	Inspection réseau	fast.log et eve.json
Wazuh Manager	Décodage, règles et alertes	événements agents et locaux
Filebeat	Transport sécurisé	alerts.json vers l'Indexer
Wazuh Indexer	Stockage et recherche	documents indexés
Wazuh Dashboard	Investigation	Threat Hunting et dashboards
Agents Wazuh	Collecte endpoint et application	EventChannel, Nginx et JSON applicatif

Segmentation logique et contrôle des flux



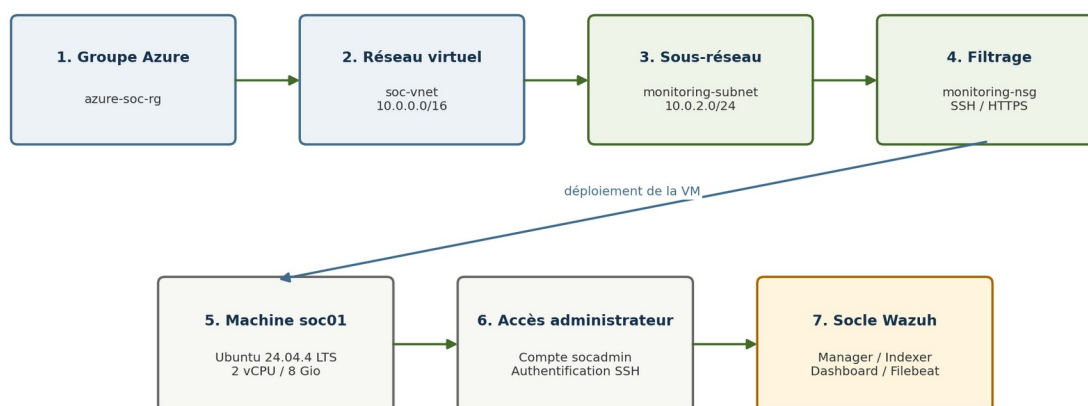
Le NSG limite les flux d'administration et conserve un refus entrant par défaut.

Figure 6 : Architecture logique du réseau Azure et filtrage NSG

3.4 Phase 1 — Infrastructure Azure et socle Wazuh

La première phase a construit la base technique sur laquelle repose l'ensemble de la supervision. L'ordre de déploiement a été volontairement progressif : organisation des ressources Azure, création du réseau, application du filtrage, déploiement de la VM, connexion SSH puis installation de Wazuh. Cette séquence permet d'identifier rapidement si une anomalie provient du Cloud, du réseau, du système Ubuntu ou d'un composant de la plateforme.

Séquence de déploiement Azure et Wazuh



Chaque ressource est validée avant l'étape suivante afin de séparer les erreurs Cloud, réseau et applicatives.

Figure 7 : Séquence de déploiement de l'infrastructure Azure et du socle Wazuh

3.4.1 Création du périmètre Azure

Le groupe de ressources azure-soc-rg a d'abord été créé dans la région France Central. Il regroupe les objets réseau et la machine virtuelle du SOC dans une même unité d'administration. Cette organisation facilite

l'inventaire des ressources, l'application des droits et le suivi de la consommation Azure. La séquence de création reprend le flux de déploiement d'une VM Linux avec clé SSH dans Azure Portal [15].

Le réseau virtuel soc-vnet utilise l'espace d'adressage 10.0.0.0/16. Le sous-réseau monitoring-subnet, défini en 10.0.2.0/24, accueille soc01. Le choix d'une adresse privée stable, 10.0.2.4, simplifie les références dans les configurations de Suricata et dans les schémas de flux. Le groupe monitoring-nsg a ensuite été associé au sous-réseau afin que le filtrage soit appliqué avant l'arrivée des paquets sur Ubuntu.

1. Dans Azure Portal, ouvrir Resource groups puis créer azure-soc-rg dans France Central.
2. Créer soc-vnet avec le préfixe 10.0.0.0/16, puis ajouter monitoring-subnet en 10.0.2.0/24.
3. Créer monitoring-nsg et l'associer à monitoring-subnet.
4. Ajouter les règles d'administration SSH et HTTPS avec des priorités inférieures aux règles Azure par défaut.

3.4.2 Déploiement de la machine virtuelle soc01

La machine soc01 a été créée avec l'image Ubuntu Server 24.04.4 LTS et le format Standard B2as v2, soit deux processeurs virtuels et 8 Gio de mémoire. L'authentification administrateur repose sur une paire de clés SSH et sur le compte socadmin. Lors de la création, l'interface réseau a été placée dans monitoring-subnet et soumise à monitoring-nsg.

Une fois le déploiement Azure terminé, la clé privée au format PEM a été conservée sur le poste d'administration. La première connexion a servi à confirmer le nom d'hôte, la version du système, l'interface eth0, l'adresse 10.0.2.4, la mémoire et l'espace disque.

```
ssh -i <chemin_cle_privée> socadmin@<adresse_publicue_masquee>
hostnamectl
ip -br address
free -h
df -h /
systemctl --failed
```

La clé a d'abord été refusée par OpenSSH sous Windows à cause de droits locaux trop larges. Les autorisations du fichier ont été restreintes au compte de l'utilisatrice, puis la connexion SSH a été répétée. La session obtenue sur soc01 a confirmé que le chemin réseau, le NSG, le compte et la clé fonctionnaient ensemble.

3.4.3 Installation all-in-one de Wazuh

Le mode all-in-one a été retenu pour installer les composants centraux sur la même VM. L'assistant officiel Wazuh installe et configure Wazuh Manager, Wazuh Indexer, Wazuh Dashboard ainsi que Filebeat. Ce choix est adapté au volume du laboratoire et limite le nombre de ressources Azure à administrer. Il ne modifie pas la séparation logique des rôles : l'analyse, l'indexation, le transport et la visualisation restent assurés par des services distincts [14].

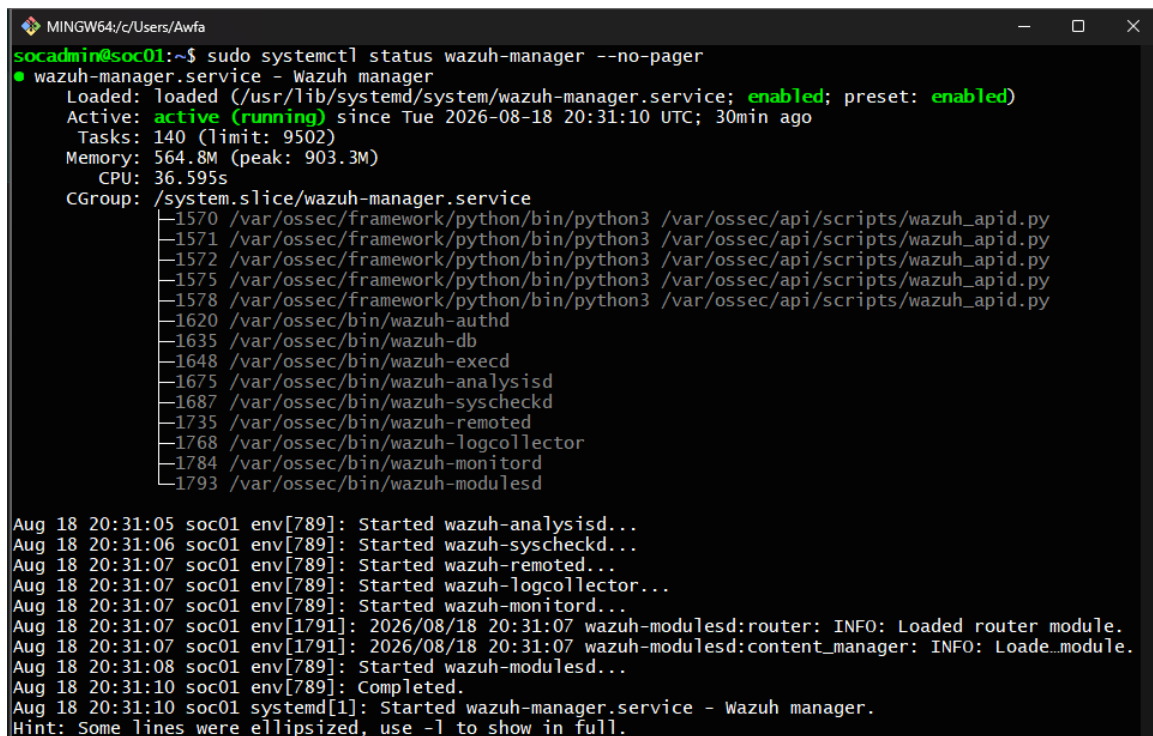
```
sudo apt update && sudo apt upgrade -y
sudo apt install curl -y
curl -sO https://packages.wazuh.com/4.14/wazuh-install.sh
sudo bash ./wazuh-install.sh -a
```

L'option -a demande à l'assistant de construire la pile complète sur l'hôte. À la fin de l'exécution, les identifiants administrateur générés ont été conservés hors du rapport. L'interface web a ensuite été ouverte en HTTPS sur le port 443 à partir d'une source autorisée par monitoring-nsg.

3.4.4 Contrôle des services Wazuh

La validation ne s'est pas limitée à l'ouverture du Dashboard. Les quatre services ont été interrogés avec systemctl, puis les ports internes ont été relus. Le Manager doit exécuter ses processus d'analyse, Logcollector et les modules internes ; l'Indexer doit accepter les connexions locales sur 9200 ; le Dashboard doit répondre sur 443 ; enfin, Filebeat doit être actif pour transporter les alertes vers l'indexeur.

```
sudo systemctl status wazuh-manager --no-pager
sudo systemctl status wazuh-indexer --no-pager
sudo systemctl status wazuh-dashboard --no-pager
sudo systemctl status filebeat --no-pager
sudo ss -lntp | grep -E ':443|:9200'
```



```
MINGW64/c/Users/Awfa
socadmin@soc01:~$ sudo systemctl status wazuh-manager --no-pager
● wazuh-manager.service - Wazuh manager
   Loaded: loaded (/usr/lib/systemd/system/wazuh-manager.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-08-18 20:31:10 UTC; 30min ago
     Tasks: 140 (limit: 9502)
    Memory: 564.8M (peak: 903.3M)
       CPU: 36.595s
    CGroup: /system.slice/wazuh-manager.service
            └─1570 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─1571 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─1572 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─1575 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─1578 /var/ossec/framework/python/bin/python3 /var/ossec/api/scripts/wazuh_apid.py
            └─1620 /var/ossec/bin/wazuh-authd
            └─1635 /var/ossec/bin/wazuh-db
            └─1648 /var/ossec/bin/wazuh-execd
            └─1675 /var/ossec/bin/wazuh-analysisd
            └─1687 /var/ossec/bin/wazuh-syscheckd
            └─1735 /var/ossec/bin/wazuh-remoted
            └─1768 /var/ossec/bin/wazuh-logcollector
            └─1784 /var/ossec/bin/wazuh-monitord
            └─1793 /var/ossec/bin/wazuh-modulesd

Aug 18 20:31:05 soc01 env[789]: Started wazuh-analysisd...
Aug 18 20:31:06 soc01 env[789]: Started wazuh-syscheckd...
Aug 18 20:31:07 soc01 env[789]: Started wazuh-remoted...
Aug 18 20:31:07 soc01 env[789]: Started wazuh-logcollector...
Aug 18 20:31:07 soc01 env[789]: Started wazuh-monitord...
Aug 18 20:31:07 soc01 env[1791]: 2026/08/18 20:31:07 wazuh-modulesd:router: INFO: Loaded router module.
Aug 18 20:31:07 soc01 env[1791]: 2026/08/18 20:31:07 wazuh-modulesd:content_manager: INFO: Loaded module.
Aug 18 20:31:08 soc01 env[789]: Started wazuh-modulesd...
Aug 18 20:31:10 soc01 env[789]: Completed.
Aug 18 20:31:10 soc01 systemd[1]: Started wazuh-manager.service - Wazuh manager.
Hint: Some lines were ellipsized, use -l to show in full.
```

Figure 8 : Service Wazuh Manager actif et sous-processus d'analyse démarrés

La Figure 8 montre Wazuh Manager à l'état actif (« active (running) » dans systemd) ainsi que plusieurs processus attendus, notamment wazuh-analysisd, wazuh-logcollector, wazuh-monitord et wazuh-modulesd. Cette lecture confirme que le service principal et ses fonctions d'analyse sont effectivement démarrés.

3.4.5 Diagnostic de la saturation du disque

Au cours de l'installation, la partition racine de 29 Go a atteint 100 % d'occupation. La saturation a perturbé les composants Wazuh et a même provoqué une erreur d'écriture lors de l'attachement à Ubuntu Pro. Un diagnostic a donc été effectué avant toute tentative de redémarrage. La commande df a d'abord confirmé l'absence d'espace disponible. L'analyse avec du a ensuite localisé environ 16 Go dans le répertoire temporaire du module de détection des vulnérabilités, sous vd_updater/tmp.

```
df -h /
sudo du -xhd1 /var | sort -h
sudo du -xhd2 /var/ossec | sort -h
sudo du -xhd3 /var/ossec/queue/vd_updater | sort -h
```

Le service concerné a été arrêté avant le nettoyage ciblé du contenu temporaire. Cette précaution a évité la suppression d'un fichier encore utilisé. Après l'opération, la partition racine est revenue à environ 45 % d'occupation, avec près de 16 Go disponibles, tandis que le cache temporaire ne représentait plus qu'environ 80 Ko.

```
sudo systemctl stop wazuh-manager
sudo find /var/ossec/queue/vd_updater/tmp/contents -mindepth 1 -delete
sudo du -sh /var/ossec/queue/vd_updater/tmp/contents
df -h /
sudo systemctl start wazuh-manager
```

Avant : partition racine à 100 %, cache vd_updater à 16 Go

```

0.0G /var/ossec/queue/vd
16G /var/ossec/queue/vd_updater
23G /var/ossec/queue
socadmin@soc01:~$ sudo du -xhd2 /var/ossec/queue/vd_updater 2>/dev/null | sort -n
312K /var/ossec/queue/vd_updater/rocksdb/updater_vulnerability_feed_manage
ta
316K /var/ossec/queue/vd_updater/rocksdb
3.6M /var/ossec/queue/vd_updater/tmp/downloads
16G /var/ossec/queue/vd_updater
16G /var/ossec/queue/vd_updater/tmp
16G /var/ossec/queue/vd_updater/tmp/contents
socadmin@soc01:~$ sudo systemctl stop wazuh-manager
socadmin@soc01:~$ sudo rm -rf /var/ossec/queue/vd_updater/tmp/contents/*
socadmin@soc01:~$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        29G   29G   340K 100% /
socadmin@soc01:~$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/root        29G   29G   320K 100% /
socadmin@soc01:~$ sudo du -sh /var/ossec/queue/vd_updater/tmp/contents
16G /var/ossec/queue/vd_updater/tmp/contents
socadmin@soc01:~$ sudo lsof +L1 | grep '/var/ossec'
socadmin@soc01:~$ sudo du -sh /var/ossec/queue/vd_updater/tmp/contents
16G /var/ossec/queue/vd_updater/tmp/contents
socadmin@soc01:~$

```

Après : racine à 45 %, 16 Go disponibles, cache à 80 Ko

```

Filesystem      Size  Used Avail Use% Mounted on
/dev/root        29G   13G   16G  45% /
socadmin@soc01:~$ sudo du -sh /var/ossec/queue/vd_updater/tmp/contents
80K /var/ossec/queue/vd_updater/tmp/contents
socadmin@soc01:~$ sudo rm -rf /var/ossec/queue/vd_updater/tmp/contents

```

Contrôle avant et après l'intervention ciblée sur le répertoire temporaire.

Figure 9 : Diagnostic et résolution de la saturation du disque

Tableau 11 : Chronologie de l'incident de capacité disque

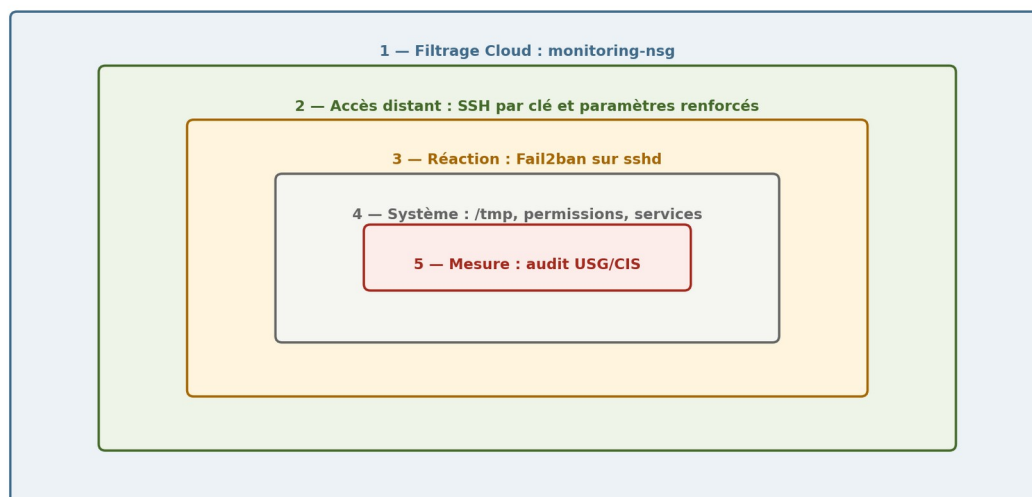
Étape	Constat	Action technique	Résultat
Détection	Partition / à 100 %	Contrôle avec df -h /	Saturation caractérisée
Localisation	vd_updater/tmp proche de 16 Go	Analyse par niveaux avec du	Répertoire responsable identifié
Sécurisation	Fichiers temporaires utilisés par un module Wazuh	Arrêt du service avant intervention	Nettoyage réalisé sans suppression large
Contrôle	Espace libéré	Nouveau contrôle avec df et du	Partition / à environ 45 % et cache à environ 80 Ko

Cet incident a montré que le stockage fait partie intégrante de la disponibilité d'un SOC. Une surveillance régulière de la partition, des index et des répertoires temporaires est aussi importante que le contrôle des services eux-mêmes.

3.5 Phase 2 — Durcissement système et réseau

La deuxième phase a renforcé soc01 avant l'activation de la détection réseau. Le travail a suivi une logique de défense en profondeur : le NSG filtre les paquets dans Azure, UFW applique une seconde politique sur l'hôte, SSH limite les méthodes et les comptes d'administration, Fail2ban réagit aux échecs répétés et les options de montage réduisent les possibilités d'abus du répertoire /tmp. Ubuntu Security Guide mesure ensuite les écarts par rapport au profil CIS Level 1 Server.

Durcissement selon une logique de défense en profondeur



Les cinq écarts CIS résiduels restent visibles : le schéma décrit les contrôles appliqués, pas une conformité totale.

Figure 10 : Défense en profondeur appliquée à soc01

3.5.1 Filtrage réseau avec Azure NSG et UFW

monitoring-nsg a été configuré pour limiter les flux entrants aux besoins d'administration. Les règles autorisent le port 22/TCP pour SSH et le port 443/TCP pour Wazuh Dashboard à partir d'adresses sources précises. Une règle SSH supplémentaire a servi au poste Kali de test. Les règles Azure par défaut s'appliquent ensuite ; DenyAllInBound, de priorité 65500, bloque tout autre flux entrant.

Network security group **monitoring-nsg** (attached to subnet: **monitoring-subnet**)
Impacts 1 subnets, 0 network interfaces

Source == all Destination == all Protocol == all Action == all Port == all + Create port rule

Prio...	Name	Port	Protocol	Source
Inbound port rules (6)				
100	Allow-SSH-MyIP	22	TCP	IP masquée
110	Allow-Wazuh-HTTPS	443	TCP	IP masquée
120	allow_Kali_Test-ssh	22	TCP	IP masquée
65000	AllowVnetInBound ⓘ	Any	Any	VirtualNetwork
65001	AllowAzureLoadBalancerInB... ⓘ	Any	Any	AzureLoadBalancer
65500	DenyAllInBound ⓘ	Any	Any	Any
Outbound port rules (3)				

Figure 11 : Règles entrantes du groupe de sécurité monitoring-nsg (adresses masquées)

La Figure 11 conserve les priorités, les noms et les ports, tout en masquant les adresses publiques. UFW complète ce contrôle dans Ubuntu. La politique entrante a été positionnée sur deny et la politique sortante sur allow. Les ports associés aux fonctions Wazuh ont ensuite été déclarés avant l'activation du pare-feu.

```

sudo ufw default deny incoming
sudo ufw default allow outgoing
sudo ufw allow OpenSSH
sudo ufw allow 443/tcp
sudo ufw enable
sudo ufw status verbose
sudo ss -tulnp

```

```

socadmin@soc01:~$ sudo ufw default allow outgoing
Default outgoing policy changed to 'allow'
(be sure to update your rules accordingly)
socadmin@soc01:~$ sudo ufw show added
Added user rules (see 'ufw status' for running firewall):
ufw allow openSSH
ufw allow 443/tcp
ufw allow 1514/tcp
ufw allow 1515/tcp
ufw allow 5601/tcp
socadmin@soc01:~$ sudo ufw default deny incoming
Default incoming policy changed to 'deny'
(be sure to update your rules accordingly)
socadmin@soc01:~$ sudo ufw show added
Added user rules (see 'ufw status' for running firewall):
ufw allow openSSH
ufw allow 443/tcp
ufw allow 1514/tcp
ufw allow 1515/tcp
ufw allow 5601/tcp
socadmin@soc01:~$ |

```

Figure 12 : Définition des règles et des politiques par défaut UFW

```

socadmin@soc01:~$ sudo ufw enable
Command may disrupt existing ssh connections. Proceed with operation (y|n)? y
Firewall is active and enabled on system startup
socadmin@soc01:~$ sudo ufw status verbose
Status: active
Logging: on (low)
Default: deny (incoming), allow (outgoing), disabled (routed)
New profiles: skip

To Action From
--
22/tcp (OpenSSH) ALLOW IN Anywhere
443/tcp ALLOW IN Anywhere
1514/tcp ALLOW IN Anywhere
1515/tcp ALLOW IN Anywhere
5601/tcp ALLOW IN Anywhere
22/tcp (OpenSSH (v6)) ALLOW IN Anywhere (v6)
443/tcp (v6) ALLOW IN Anywhere (v6)
1514/tcp (v6) ALLOW IN Anywhere (v6)
1515/tcp (v6) ALLOW IN Anywhere (v6)
5601/tcp (v6) ALLOW IN Anywhere (v6)

```

Figure 13 : Activation et état du pare-feu UFW sur soc01

La lecture des Figures 12 et 13 montre deux niveaux différents : les règles UFW sont générales sur l'hôte, tandis que le NSG restreint les sources avant la VM. Les ports 1514 et 1515 correspondent aux communications d'agents Wazuh ; le port 443 dessert le Dashboard. Le contrôle final porte à la fois sur la politique par défaut, la journalisation UFW et la liste des sockets réellement en écoute.

3.5.2 Durcissement du service SSH

L'administration distante constitue un point d'entrée sensible. La configuration SSH a été renforcée en interdisant la connexion directe de root, en désactivant l'authentification par mot de passe, en privilégiant les

clés et en limitant les comptes autorisés avec AllowUsers socadmin. Le nombre de tentatives, le délai de connexion et le nombre de sessions simultanées ont aussi été contrôlés.

```
sudo sshd -t
sudo sshd -T | grep -E 'permitrootlogin|passwordauthentication|pubkeyauthentication|allowusers|
maxauthtries|clntalive'
```

La commande sshd -t a été exécutée avant le rechargement du service. Ce contrôle de syntaxe est indispensable, car une erreur dans sshd_config peut interrompre l'accès distant à la VM. Les valeurs effectives ont ensuite été relues avec sshd -T.

```
socadmin@soc01:~$ sudo sshd -T | grep -E 'passwordauthentication|pubkeyauthentication'
pubkeyauthentication yes
passwordauthentication no
```

Figure 14 : Authentification SSH par clé et désactivation du mot de passe

```
Rule    xccdf_org.ssgproject.content_rule_sshd_limit_user_access
Result  pass

Title   Ensure SSH LoginGraceTime is configured
Rule    xccdf_org.ssgproject.content_rule_sshd_set_login_grace_time
Result  pass

Title   Set LogLevel to INFO
Rule    xccdf_org.ssgproject.content_rule_sshd_set_loglevel_info
Result  pass

Title   Set SSH authentication attempt limit
Rule    xccdf_org.ssgproject.content_rule_sshd_set_max_auth_tries
Result  pass

Title   Set SSH MaxSessions limit
Rule    xccdf_org.ssgproject.content_rule_sshd_set_max_sessions
Result  pass

Title   Ensure SSH MaxStartups is configured
Rule    xccdf_org.ssgproject.content_rule_sshd_set_maxstartups
Result  pass

Title   Use Only Strong Ciphers
Rule    xccdf_org.ssgproject.content_rule_sshd_use_strong_ciphers
Result  pass

Title   Use Only Strong Key Exchange algorithms
Rule    xccdf_org.ssgproject.content_rule_sshd_use_strong_kex
Result  pass

Title   Use Only Strong MACs
Rule    xccdf_org.ssgproject.content_rule_sshd_use_strong_macs
Result  pass

USG audit scan command completed. The scan results are available in /var/lib/usg/usg-report-20260816.1556.html report or in /var/lib/usg/usg-results-20260816.1556.xml
socadmin@soc01:~$
```

Figure 15 : Contrôles de durcissement SSH issus de l'audit USG

La Figure 14 confirme que pubkeyauthentication est activé et que passwordauthentication est désactivé. La Figure 15 complète ce contrôle par les résultats USG relatifs aux limites de session et aux algorithmes cryptographiques. Une seconde session SSH a été ouverte avant de fermer la première afin de valider l'accès après le rechargement.

3.5.3 Protection contre les tentatives répétées

Fail2ban a été activé pour surveiller les échecs d'authentification SSH. Lorsqu'une même adresse dépasse le seuil défini, la jail sshd applique un bannissement temporaire via nftables. Le fichier jail.local fixe une fenêtre d'observation de dix minutes, cinq essais au maximum et un bannissement initial de quinze minutes. L'incrémentation du bantime augmente la durée pour les récidives.

```
socadmin@soc01: /etc/fail2ban
GNU nano 7.2
[DEFAULT]

# Never ban localhost
ignoreip = 127.0.0.1/8 ::1

# Basic ban policy
bantime = 15m
findtime = 10m
maxretry = 5

# Increase ban duration for repeat offenders
bantime.increment = true
bantime.multipliers = 1 2 4 8 16 32 64 96
bantime.maxtime = 24h

# General log backend
backend = auto

# Avoid DNS resolution when banning
usedns = no

# Ban action
banaction = nftables
banaction_allports = nftables[type=allports]

# Ban only, no email from Fail2ban
action = %(action_)s

[sshd]

enabled = true
port = ssh
filter = sshd
backend = systemd
mode = normal
```

Figure 16 : Configuration de la jail sshd dans le fichier jail.local

```
sudo fail2ban-client -t
sudo systemctl restart fail2ban
sudo fail2ban-client status
sudo fail2ban-client status sshd
sudo journalctl -u fail2ban --since '10 minutes ago'
```

```
fail2ban.conf fail2ban.local jail.conf paths-arch.conf paths-debian.conf
socadmin@soc01:/etc/fail2ban$ sudo nano jail.local
socadmin@soc01:/etc/fail2ban$ sudo fail2ban-client -t
2026-08-11 16:18:20.603 fail2ban.configreader [6302]: WARNING 'allowip6' not defined in 'Definition'. Using default one: 'auto'
OK: configuration test is successful
socadmin@soc01:/etc/fail2ban$ sudo systemctl restart fa
factory-reset.target fail2ban.service
socadmin@soc01:/etc/fail2ban$ sudo systemctl restart fail2ban
socadmin@soc01:/etc/fail2ban$ sudo fail2ban-client status sshd
Status for the jail: sshd
|- Filter
| |- Currently failed: 0
| |- Total failed: 0
| - Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
- Actions
| |- Currently banned: 0
| |- Total banned: 0
| - Banned IP list:
socadmin@soc01:/etc/fail2ban$
```

Figure 17 : Validation de la configuration et de la jail sshd de Fail2ban

```
socadmin@soc01:~$ sudo systemctl status fail2ban --no-pager
● fail2ban.service - Fail2Ban Service
   Loaded: loaded (/usr/lib/systemd/system/fail2ban.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-08-11 15:46:48 UTC; 8min ago
     Docs: man:fail2ban(1)
   Main PID: 5675 (fail2ban-server)
      Tasks: 5 (limit: 9490)
    Memory: 20.5M (peak: 22.6M)
       CPU: 527ms
    CGroup: /system.slice/fail2ban.service
            └─5675 /usr/bin/python3 /usr/bin/fail2ban-server -xf start

Aug 11 15:46:48 soc01 systemd[1]: Started fail2ban.service - Fail2Ban Service.
Aug 11 15:46:48 soc01 fail2ban-server[5675]: 2026-08-11 15:46:48,331 fail2ban.configreader [5675]: W
Aug 11 15:46:48 soc01 fail2ban-server[5675]: Server ready
Hint: Some lines were ellipsized, use -l to show in full.
socadmin@soc01:~$ cd /etc/fail2ban/
```

Figure 18 : Service Fail2ban actif sur soc01

Le test de configuration retourne OK avant le redémarrage. Le statut systemd confirme ensuite que le serveur Fail2ban est prêt. Un test SSH contrôlé a produit dix échecs cumulés ; une adresse était bannie au moment du contrôle et deux bannissements avaient été enregistrés au total.

```
socadmin@soc01:~$ sudo fail2ban-client status sshd
status for the jail: sshd
- Filter
  |- Currently failed: 0
  |- Total failed:    10
  '- Journal matches: _SYSTEMD_UNIT=sshd.service + _COMM=sshd
- Actions
  |- Currently banned: 1
  |- Total banned:
  '- Banned IP list:
```

Figure 19 : Réaction de Fail2ban lors du test SSH (adresse masquée)

La Figure 19 illustre la réaction de la jail sshd pendant le test. L'adresse source a été masquée pour éviter d'exposer une information d'accès, tout en conservant le nombre de tentatives et l'état du bannissement.

3.5.4 Sécurisation de /tmp et des permissions

Le répertoire /tmp a été monté en tmpfs avec les options nosuid, nodev et noexec. nosuid empêche la prise en compte des bits SUID et SGID, nodev interdit l'utilisation de fichiers spéciaux de périphérique et noexec bloque l'exécution directe de fichiers depuis ce point de montage. Ces options réduisent l'intérêt de /tmp comme espace de dépôt pour un programme malveillant.

```
sudo cp /etc/fstab /etc/fstab.backup-before-cis-tmp
echo 'tmpfs /tmp tmpfs rw,nodev,nosuid,noexec,relatime,size=1G 0 0' | sudo tee -a /etc/fstab
sudo systemctl daemon-reload
sudo mount -a
findmnt /tmp
```

```
MINGW64/c/Users/Awfa
socadmin@soc01:~$ findmnt /tmp
TARGET
SOURCE
FSTYPE OPTIONS
/tmp tmpfs tmpfs rw,nosuid,nodev,noexec,relatime,size=1048576k,inode64
socadmin@soc01:~$
```

Figure 20 : Options de montage sécurisées appliquées à /tmp

Les permissions trop ouvertes identifiées sous /var/log ont été recherchées avec find, puis corrigées sur les fichiers concernés. Un fichier drop-in SSH a également été créé pour appliquer AllowUsers socadmin. La syntaxe a été testée avant le rechargement du service.

```
sudo find /var/log -type f -perm /0137 -printf '%m %u:%g %p\n'
echo 'AllowUsers socadmin' | sudo tee /etc/ssh/ssh_config.d/99-allowusers.conf
```

```
sudo chmod 600 /etc/ssh/sshd_config.d/99-allowusers.conf
sudo sshd -t && sudo systemctl reload ssh
```

```
socadmin@soc01:~$ sudo chmod o-r \
/var/log/apt/history.log \
/var/log/sysstat/sa16 \
/var/log/waagent.log \
/var/log/wazuh-indexer/gc.log \
/var/log/wazuh-indexer/gc.log.{20,21,22,23,24}
socadmin@soc01:~$ sudo find /var/log -type f \
! -path '/var/log/wtmp' \
! -path '/var/log/lastlog' \
! -path '/var/log/btmp' \
-perm /0137 \
-printf '%m %u:%g %p\n'
socadmin@soc01:~$ sudo sshd -T | grep -E '^(allowusers|allowgroups|denyusers|denygroups)'
socadmin@soc01:~$ echo 'AllowUsers socadmin' | sudo tee /etc/ssh/sshd_config.d/99-allowu
sers.conf
AllowUsers socadmin
socadmin@soc01:~$ sudo sshd -t
socadmin@soc01:~$ sudo sshd -T | grep '^allowusers'
allowusers socadmin
socadmin@soc01:~$ sudo systemctl reload ssh
socadmin@soc01:~$
```

Figure 21 : Correction des permissions de journaux et restriction de l'accès SSH à socadmin

3.5.5 Évaluation avec Ubuntu Security Guide

La première tentative d'activation a échoué parce que la VM n'était pas attachée à un abonnement Ubuntu Pro. pro status affichait le service USG comme disponible, mais pro enable usg retournait Cannot enable unknown service. Après l'attachement de la machine et la libération d'espace disque, le dépôt USG a été activé et le paquet installé [9][16].

```
socadmin@soc01:~$ sudo pro status
SERVICE      AVAILABLE DESCRIPTION
anbox-cloud    yes      Scalable Android in the cloud
esm-apps       yes      Expanded Security Maintenance for Applications
esm-infra      yes      Expanded Security Maintenance for Infrastructure
fips-updates   yes      FIPS compliant crypto packages with stable security updat
landscape      yes      Management and administration tool for Ubuntu
livepatch      yes      Canonical Livepatch service
realtime-kernel yes      Ubuntu kernel with PREEMPT_RT patches integrated
usg            yes      Security compliance and audit tools

For a list of all Ubuntu Pro services, run 'pro status --all'

This machine is not attached to an Ubuntu Pro subscription.
See https://ubuntu.com/pro
socadmin@soc01:~$ sudo pro enable usg
Cannot enable unknown service 'usg'.
```

Figure 22 : État initial d'Ubuntu Pro avant l'activation de USG

```
sudo pro status
sudo pro attach
sudo pro enable usg
sudo apt install usg
```

Ubuntu Security Guide a ensuite été utilisé avec le profil cis_level1_server. Le premier audit faisait apparaître 106 règles en échec. Après les remédiations, un ré-audit en a compté 6, puis le dernier contrôle en a conservé 5. Cette baisse traduit une amélioration importante de la configuration ; elle ne correspond pas à une conformité CIS totale.

```
sudo usg audit cis_level1_server
sudo grep -o '<result>fail</result>' /var/lib/usg/usg-results-<horodatage>.xml | wc -l
```

```
socadmin@soc01:~$ sudo grep -o '<result>fail</result>' /var/lib/usg
/usg-results-20260816.1439.xml | wc -l
6
socadmin@soc01:~$
```

Figure 23 : Ré-audit CIS intermédiaire avec six règles en échec

```
socadmin@soc01:~$ sudo python3 - <<'PY'
import xml.etree.ElementTree as ET

file = "/var/lib/usg/usg-results-20260816.1439.xml"
root = ET.parse(file).getroot()

for item in root.iter():
    if item.tag.endswith("rule-result"):
        result = next(
            (child.text for child in item if child.tag.endswith("result")),
            None
        )
        if result == "fail":
            print(item.attrib.get("idref"))
PY
xccdf_org.ssgproject.content_rule_partition_for_tmp
xccdf_org.ssgproject.content_rule_all_apparmor_profiles_in_enforce_complain_mode
xccdf_org.ssgproject.content_rule_grub2_uefi_password
xccdf_org.ssgproject.content_rule_nftables_ensure_default_deny_policy
xccdf_org.ssgproject.content_rule_permissions_local_var_log
xccdf_org.ssgproject.content_rule_sshd_limit_user_access
socadmin@soc01:~$
```

Figure 24 : Extraction des six règles CIS restant à traiter

L'extraction du rapport XML a fourni les identifiants exacts des contrôles restant à traiter. La règle `partition_for_tmp` a disparu après le montage séparé de `/tmp` ; la restriction `AllowUsers` a traité le contrôle d'accès SSH, tandis que les permissions de journaux et des fichiers drop-in ont fait l'objet de corrections ciblées.

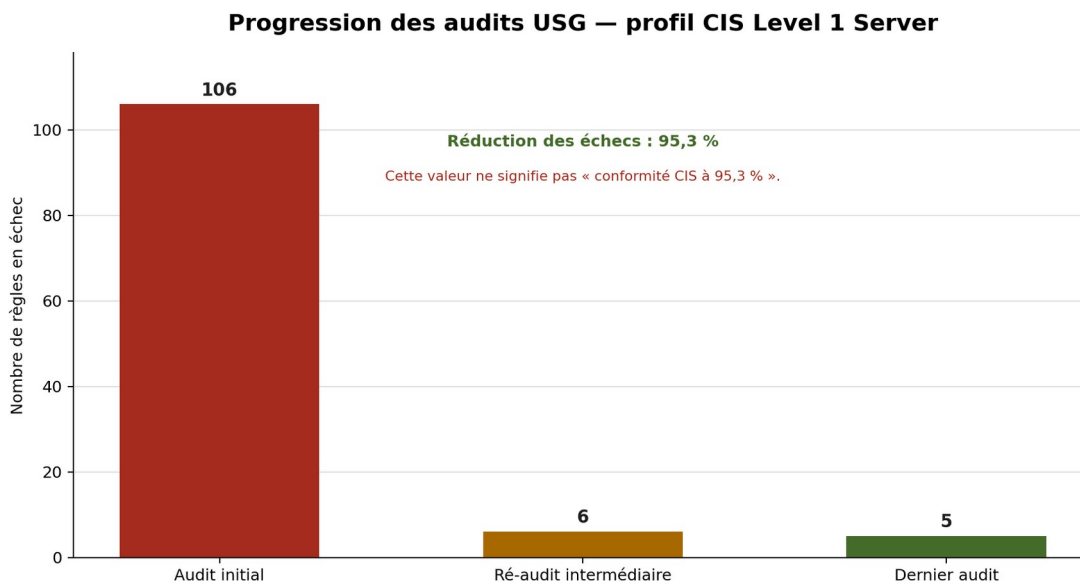


Figure 25 : Évolution du nombre de règles CIS en échec

```

ocadmin@soc01:~$ sudo python3 - <<'PY'
import xml.etree.ElementTree as ET

file = "/var/lib/usg/usg-results-20260816.1556.xml"
root = ET.parse(file).getroot()

for item in root.iter():
    if item.tag.endswith("rule-result"):
        result = next(
            (child.text for child in item if child.tag.endswith("result")),
            None
        )
        if result == "fail":
            print(item.attrib.get("idref"))

ccdf_org.ssgproject.content_rule_all_apparmor_profiles_in_enforce_complain_mode
ccdf_org.ssgproject.content_rule_grub2_uefi_password
ccdf_org.ssgproject.content_rule_nftables_ensure_default_deny_policy
ccdf_org.ssgproject.content_rule_permissions_local_var_log
ccdf_org.ssgproject.content_rule_file_permissions_sshd_drop_in_config
ocadmin@soc01:~$

```

Figure 26 : Cinq écarts CIS résiduels après durcissement

Tableau 12 : Écarts CIS résiduels

Domaine	Règle en échec	Interprétation technique
AppArmor	all_apparmor_profiles_in_enforce_complain_mode	Identifier le profil qui n'utilise ni le mode enforce ni le mode complain
Démarrage	grub2_uefi_password	Le contrôle CIS attend la protection de GRUB par mot de passe UEFI
Pare-feu local	nftables_ensure_default_deny_policy	La politique locale nftables ne satisfait pas le refus par défaut attendu
Journaux	permissions_local_var_log	Un ou plusieurs objets de /var/log présentent des droits à corriger
SSH	file_permissions_sshd_drop_in_config	Les permissions des fichiers de sshd_config.d doivent être harmonisées

Tableau 13 : Contrôles de durcissement appliqués

Couche	Mesure appliquée	Contrôle réalisé
Azure	Règles NSG ciblées et DenyAllInBound	Lecture des priorités, sources et ports
Pare-feu local	UFW actif	ufw status verbose et ports en écoute
SSH	Clés, root interdit, mot de passe désactivé, AllowUsers	ssh -t puis sshd -T
Réaction	Jail Fail2ban pour sshd	Tentatives contrôlées et hôte banni
Système de fichiers	/tmp avec nosuid,nodev,noexec	findmnt /tmp
Référentiel	Audit CIS Level 1 Server	Évolution de 106 à 5 règles en échec

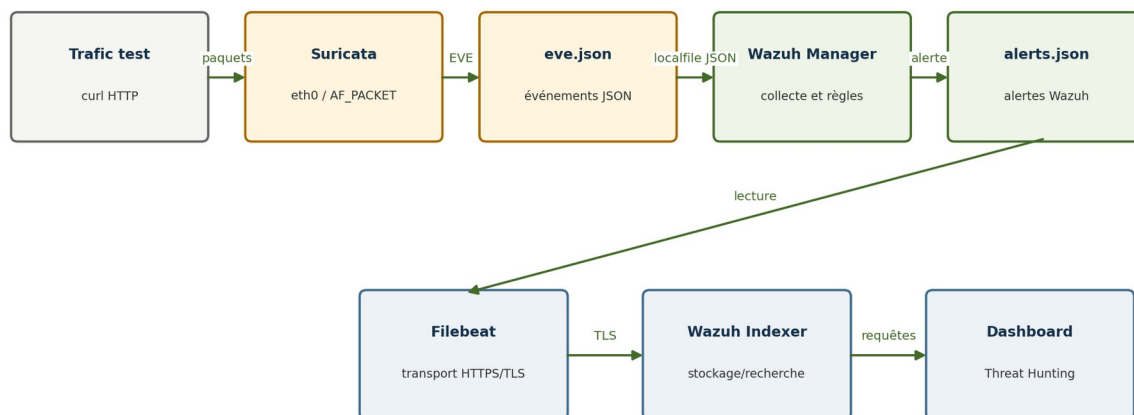
Le résultat final conserve cinq écarts : un profil AppArmor, la protection GRUB UEFI, la politique locale nftables, des permissions sous /var/log et les droits d'un fichier SSH drop-in. Le rapport les maintient visibles afin de distinguer le durcissement effectivement appliqué d'une certification CIS.

3.6 Phase 3 — Intégration Suricata/Wazuh

La troisième phase a ajouté la détection réseau au socle Wazuh. L'objectif technique était de suivre un même événement depuis le paquet analysé sur eth0 jusqu'à sa consultation dans Threat Hunting. La Figure 27 représente les maillons successifs et les fichiers ou protocoles utilisés entre eux. Cette décomposition est

essentielle : si une alerte n'apparaît pas dans le Dashboard, chaque niveau peut être testé sans modifier les autres.

Chaîne de détection et d'indexation Suricata/Wazuh



Le test HTTP déclenche le SID 2100498 ; les contrôles suivants vérifient la collecte, le transport et l'affichage Wazuh.

Figure 27 : Chaîne technique de détection et d'indexation Suricata/Wazuh

Tableau 14 : Paramètres de fonctionnement de Suricata

Paramètre	Valeur	Rôle
Version	Suricata 7.0.3	Moteur de détection réseau installé sur soc01
Interface	eth0	Interface inspectée par le service
HOME_NET	192.168.0.0/16 ; 10.0.0.0/8 ; 172.16.0.0/12	Plages privées considérées comme internes
Mode d'acquisition	AF_PACKET	Capture du trafic sur l'interface Linux
Fichier rapide	/var/log/suricata/fast.log	Lecture immédiate des signatures
Fichier structuré	/var/log/suricata/eve.json	Source JSON collectée par Wazuh
Signature de test	SID 2100498	Identification du scénario HTTP contrôlé

3.6.1 Installation de Suricata

Suricata et son outil de mise à jour ont été installés sur soc01 à partir des dépôts Ubuntu. La version relevée par le test de configuration est 7.0.3. Avant toute modification, le fichier YAML a été sauvegardé afin de disposer d'un retour arrière simple.

```

sudo apt update
sudo apt install suricata suricata-update -y
sudo cp /etc/suricata/suricata.yaml /etc/suricata/suricata.yaml.backup
suricata --build-info | head

```

3.6.2 Configuration de l'interface et de HOME_NET

La commande `ip -br address` a montré que `eth0` porte l'adresse privée `10.0.2.4/24`. Cette interface a donc été déclarée dans la section `af-packet` du fichier `suricata.yaml`. `AF_PACKET` permet à Suricata de capturer les paquets directement dans le noyau Linux.

```

ip -br address
sudo grep -A 8 '^af-packet:' /etc/suricata/suricata.yaml

```

```

socadmin@soc01:~$ ip -br address
lo                UNKNOWN      127.0.0.1/8 ::1/128
eth0              UP            10.0.2.4/24 metric 100 fe80::7eed:8dff:fe6d:3638/64
socadmin@soc01:~$ sudo grep -A 8 '^af-packet:' /etc/suricata/suricata.yaml
af-packet:
- interface: eth0
  # Number of receive threads. "auto" uses the number of cores
  #threads: auto
  # Default clusterid. AF_PACKET will load balance packets based on flow.
  cluster-id: 99
  # Default AF_PACKET cluster type. AF_PACKET can load balance per flow or per hash.
  # This is only supported for Linux kernel > 3.1
  # possible value are:
socadmin@soc01:~$

```

Figure 28 : Interface eth0 sélectionnée pour l'acquisition AF_PACKET

HOME_NET a été défini avec les trois plages privées RFC 1918 : 192.168.0.0/16, 10.0.0.0/8 et 172.16.0.0/12. Le réseau Azure 10.0.0.0/16 appartient à la deuxième plage. Cette configuration couvre le laboratoire ; une exploitation plus restrictive pourrait ramener HOME_NET au seul préfixe Azure afin de réduire les correspondances inutiles.

```
sudo grep -n 'HOME_NET:' /etc/suricata/suricata.yaml | head
```

```

socadmin@soc01:~$ sudo grep -n 'HOME_NET:' /etc/suricata/suricata.yaml | head
18: HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"
19: #HOME_NET: "[192.168.0.0/16]"
20: #HOME_NET: "[10.0.0.0/8]"
21: #HOME_NET: "[172.16.0.0/12]"
22: #HOME_NET: "any"
socadmin@soc01:~$

```

Figure 29 : Plages réseau déclarées dans HOME_NET

3.6.3 Installation des règles Emerging Threats

Le premier test de configuration a signalé l'absence du fichier de règles attendu. Le paquet Suricata était installé, mais le moteur ne pouvait pas charger une politique de détection complète. La commande `suricata -T` indiquait explicitement qu'aucun fichier ne correspondait à `/var/lib/suricata/rules/suricata.rules`.

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
```

```

socadmin@soc01:~$ sudo suricata -T -c /etc/suricata/suricata.yaml
i: suricata: This is Suricata version 7.0.3 RELEASE running in SYSTEM mode
W: detect: No rule files match the pattern /var/lib/suricata/rules/suricata.rules
socadmin@soc01:~$

```

Figure 30 : Échec initial du test Suricata en l'absence de fichier de règles

`suricata-update` a ensuite téléchargé, fusionné et activé le jeu Emerging Threats. La sortie fait apparaître environ 52 360 règles activées. Le test de configuration a été relancé avant le redémarrage, ce qui sépare une erreur de signatures d'un problème de service.

```

sudo suricata-update
sudo suricata -T -c /etc/suricata/suricata.yaml -v
sudo systemctl restart suricata

```

```

socadmin@soc01:~$ sudo suricata-update
18/8/2026 -- 20:48:18 - <Info> -- Using data-directory /var/lib/suricata.
18/8/2026 -- 20:48:18 - <Info> -- Using Suricata configuration /etc/suricata/suricata.yaml
18/8/2026 -- 20:48:18 - <Info> -- Using /etc/suricata/rules for Suricata provided rules.
18/8/2026 -- 20:48:18 - <Info> -- Found Suricata version 7.0.3 at /usr/bin/suricata.
18/8/2026 -- 20:48:18 - <Info> -- Loading /etc/suricata/suricata.yaml
18/8/2026 -- 20:48:18 - <Info> -- Disabling rules for protocol pgsql
18/8/2026 -- 20:48:18 - <Info> -- Disabling rules for protocol modbus
18/8/2026 -- 20:48:18 - <Info> -- Disabling rules for protocol dnp3
18/8/2026 -- 20:48:18 - <Info> -- Disabling rules for protocol enip
18/8/2026 -- 20:48:18 - <Info> -- No sources configured, will use Emerging Threats Open
18/8/2026 -- 20:48:18 - <Info> -- Fetching https://rules.emergingthreats.net/open/suricata-7.0.3/emerging.rules.tar.gz.
100% - 5581145/5581145
18/8/2026 -- 20:48:20 - <Info> -- Done.
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/app-layer-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/decoder-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/dhcp-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/dnp3-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/dns-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/files.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/http-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/ipsec-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/kerberos-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/modbus-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/nfs-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/ntp-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/smb-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/smtp-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/stream-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Loading distribution rule file /etc/suricata/rules/tls-events.rules
18/8/2026 -- 20:48:20 - <Info> -- Ignoring file rules/emerging-deleted.rules
18/8/2026 -- 20:48:23 - <Info> -- Loaded 68301 rules.
18/8/2026 -- 20:48:23 - <Info> -- Disabled 14 rules.
18/8/2026 -- 20:48:23 - <Info> -- Enabled 0 rules.
18/8/2026 -- 20:48:23 - <Info> -- Modified 0 rules.
18/8/2026 -- 20:48:23 - <Info> -- Dropped 0 rules.
18/8/2026 -- 20:48:23 - <Info> -- Enabled 136 rules for flowbit dependencies.
18/8/2026 -- 20:48:23 - <Info> -- Backing up current rules.
18/8/2026 -- 20:48:23 - <Info> -- Writing rules to /var/lib/suricata/rules/suricata.rules: total: 68301; enabled: 52360; added: 68301; removed 0; modified: 0
18/8/2026 -- 20:48:23 - <Info> -- Writing /var/lib/suricata/rules/classification.config
18/8/2026 -- 20:48:24 - <Info> -- Testing with suricata -T.
18/8/2026 -- 20:48:54 - <Info> -- Done.
socadmin@soc01:~$

```

Figure 31 : Téléchargement et activation des règles Emerging Threats

```

socadmin@soc01:~$ sudo tail -n 25 /var/log/suricata/suricata.log
[6767 - Suricata-Main] 2026-08-18 20:47:34 Info: suricata: Setting engine mode to IDS mode by default
[6767 - Suricata-Main] 2026-08-18 20:47:34 Info: exception-policy: master exception-policy set to: auto
[6767 - Suricata-Main] 2026-08-18 20:47:34 Info: logopenfile: fast output device (regular) initialized:
Fast.log
[6767 - Suricata-Main] 2026-08-18 20:47:34 Info: logopenfile: eve-log output device (regular) initialize
d: eve.json
[6767 - Suricata-Main] 2026-08-18 20:47:34 Info: logopenfile: stats output device (regular) initialized:
stats.log
[6767 - Suricata-Main] 2026-08-18 20:47:34 Warning: detect: No rule files match the pattern /var/lib/sur
icata/rules/suricata.rules
[5225 - Suricata-Main] 2026-08-18 20:50:48 Notice: suricata: Signal Received. Stopping engine.
[5225 - Suricata-Main] 2026-08-18 20:50:49 Info: suricata: time elapsed 696.785s
[5225 - Suricata-Main] 2026-08-18 20:50:50 Info: counters: Alerts: 0
[5225 - Suricata-Main] 2026-08-18 20:50:50 Notice: device: eth0: packets: 16735, drops: 0 (0.00%), inval
id checksum: 0
[5225 - Suricata-Main] 2026-08-18 20:50:50 Warning: ioctl: eth0: failed to set SIOCETHTOOL ioctl: Invali
d argument
[6798 - Suricata-Main] 2026-08-18 20:50:50 Notice: suricata: This is Suricata version 7.0.3 RELEASE runn
ing in SYSTEM mode
[6798 - Suricata-Main] 2026-08-18 20:50:50 Info: cpu: CPUs/cores online: 2
[6798 - Suricata-Main] 2026-08-18 20:50:50 Info: suricata: Setting engine mode to IDS mode by default
[6798 - Suricata-Main] 2026-08-18 20:50:50 Info: exception-policy: master exception-policy set to: auto
[6798 - Suricata-Main] 2026-08-18 20:50:50 Info: ioctl: eth0: MTU 1500
[6801 - Suricata-Main] 2026-08-18 20:50:50 Info: logopenfile: fast output device (regular) initialized:
Fast.log
[6801 - Suricata-Main] 2026-08-18 20:50:50 Info: logopenfile: eve-log output device (regular) initialize
d: eve.json
[6801 - Suricata-Main] 2026-08-18 20:50:50 Info: logopenfile: stats output device (regular) initialized:
stats.log
[6801 - Suricata-Main] 2026-08-18 20:51:02 Info: detect: 1 rule files processed. 52360 rules successfull
y loaded, 0 rules failed, 0
[6801 - Suricata-Main] 2026-08-18 20:51:02 Info: threshold-config: Threshold config parsed: 0 rule(s) fo
und
[6801 - Suricata-Main] 2026-08-18 20:51:02 Info: detect: 52365 signatures processed. 1329 are IP-only ru
les, 4511 are inspecting packet payload, 46292 inspect application layer, 108 are decoder event only
[6801 - Suricata-Main] 2026-08-18 20:51:22 Info: runmodes: eth0: creating 2 threads
[6801 - Suricata-Main] 2026-08-18 20:51:22 Info: unix-manager: unix socket '/var/run/suricata-command.so
cket'
[6801 - Suricata-Main] 2026-08-18 20:51:22 Notice: threads: Threads created -> W: 2 FM: 1 FR: 1 Engine
started.
socadmin@soc01:~$

```

Figure 32 : Chargement des règles et initialisation des journaux Suricata

La règle utilisée pour la validation de bout en bout a été recherchée directement dans le fichier chargé. La présence de sid:2100498 confirme que le moteur dispose de la signature GPL ATTACK_RESPONSE id check returned root avant l'envoi du trafic.

```
sudo grep -n 'sid:2100498;' /var/lib/suricata/rules/suricata.rules
```

```
socadmin@soc01:~$ sudo grep -n 'sid:2100498;' /var/lib/suricata/rules/suricata.rules
2597:alert ip any any -> any any:"GPL ATTACK_RESPONSE id check returned root"; content:"
; sid:2100498; rev:7; metadata:created_at 2010_09_23, confidence Medium, signature_severity I
socadmin@soc01:~$
```

Figure 33 : Présence de la signature SID 2100498 dans le jeu de règles

3.6.4 Activation du capteur et contrôle des journaux

Le service a été associé à l'interface eth0 en mode AF_PACKET. Après le redémarrage, systemctl a confirmé son état active (running). La commande du processus reprend l'option -i eth0, ce qui confirme que le service ne fonctionne pas simplement en arrière-plan, mais qu'il observe bien l'interface attendue.

```
sudo systemctl status suricata --no-pager
sudo journalctl -u suricata -n 50 --no-pager
sudo ls -lh /var/log/suricata/fast.log /var/log/suricata/eve.json
```

```
socadmin@soc01:~$ sudo systemctl restart suricata
socadmin@soc01:~$ sudo systemctl status suricata --no-pager
● suricata.service - Suricata IDS/IDP daemon
   Loaded: loaded (/usr/lib/systemd/system/suricata.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-08-18 20:50:50 UTC; 30s ago
     Docs: man:suricata(8)
           man:suricatasc(8)
           https://suricata.io/documentation/
  Process: 6798 ExecStart=/usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suricata.pid (code=exited, status=0/SUCCESS)
    Main PID: 6801 (Suricata-Main)
      Tasks: 1 (limit: 9502)
     Memory: 497.7M (peak: 497.7M)
        CPU: 29.754s
      CGroup: /system.slice/suricata.service
             └─6801 /usr/bin/suricata -D --af-packet -c /etc/suricata/suricata.yaml --pidfile /run/suri...

Aug 18 20:50:50 soc01 systemd[1]: Starting suricata.service - Suricata IDS/IDP daemon...
Aug 18 20:50:50 soc01 suricata[6798]: i: suricata: This is Suricata version 7.0.3 RELEASE running_M mode
Aug 18 20:50:50 soc01 systemd[1]: Started suricata.service - Suricata IDS/IDP daemon.
Hint: Some lines were ellipsized, use -l to show in full.
socadmin@soc01:~$
```

Figure 34 : Service Suricata actif sur l'interface eth0

eve.json a ensuite été lu avec tail. Les lignes contiennent un horodatage, l'interface, les adresses, le protocole et le type d'événement. Cette sortie prouve que le capteur produit des données JSON structurées exploitables par la chaîne Wazuh.

```
sudo tail -n 3 /var/log/suricata/eve.json
```

```
socadmin@soc01:~$ sudo tail -n 3 /var/log/suricata/eve.json
{"timestamp":"2026-08-18T20:53:06.056175+0000","flow_id":804222927315650,"in_iface":"eth0","event_type":
"dns","src_ip":"10.0.2.4","src_port":42459,"dest_ip":"168.63.129.16","dest_port":53,"proto":"UDP","pkt_s
rc":"wire/pcap","dns":{"type":"query","id":53401,"rrname":"soc01.zngk01hknf4eIntomluif5xnqq.parx.interna
l.cloudapp.net","rrtype":"AAAA","tx_id":0,"opcode":0}}
{"timestamp":"2026-08-18T20:53:06.059246+0000","flow_id":802843455822420,"in_iface":"eth0","event_type":
"dns","src_ip":"10.0.2.4","src_port":57408,"dest_ip":"168.63.129.16","dest_port":53,"proto":"UDP","pkt_s
rc":"wire/pcap","dns":{"version":2,"type":"answer","id":8493,"flags":"8180","qr":true,"rd":true,"ra":tru
e,"opcode":0,"rrname":"soc01.zngk01hknf4eIntomluif5xnqq.parx.internal.cloudapp.net","rrtype":"A","rcode"
:"NOERROR","answers":[{"rrname":"soc01.zngk01hknf4eIntomluif5xnqq.parx.internal.cloudapp.net","rrtype":"
A","ttl":10,"rdata":"10.0.2.4"}],"grouped":{"A":["10.0.2.4"]}}}
{"timestamp":"2026-08-18T20:53:06.060876+0000","flow_id":804222927315650,"in_iface":"eth0","event_type":
"dns","src_ip":"10.0.2.4","src_port":42459,"dest_ip":"168.63.129.16","dest_port":53,"proto":"UDP","pkt_s
rc":"wire/pcap","dns":{"version":2,"type":"answer","id":53401,"flags":"8180","qr":true,"rd":true,"ra":tr
ue,"opcode":0,"rrname":"soc01.zngk01hknf4eIntomluif5xnqq.parx.internal.cloudapp.net","rrtype":"AAAA","rc
ode":"NOERROR","authorities":[{"rrname":"internal.cloudapp.net","rrtype":"SOA","ttl":10,"soa":{"mname":"
azureprivatedns.net","rname":"azureprivatedns-host.microsoft.com","serial":1,"refresh":900,"retry":600,"
expire":86400,"minimum":10}}]}}}
socadmin@soc01:~$
```

Figure 35 : Production d'événements structurés dans eve.json

3.6.5 Intégration du journal EVE à Wazuh

Avant la modification, `ossec.conf` a été copié vers un fichier de sauvegarde. Un bloc `localfile` a ensuite été ajouté pour surveiller `/var/log/suricata/eve.json` avec le format json. Ce choix permet au décodeur JSON de conserver les champs de l'événement, au lieu de traiter toute la ligne comme une chaîne de caractères [7].

```
sudo cp /var/ossec/etc/ossec.conf /var/ossec/etc/ossec.conf.backup-before-suricata
```

```
<localfile>  
<log_format>json</log_format>  
<location>/var/log/suricata/eve.json</location>  
</localfile>
```

```
<ossec_config>  
  <localfile>  
    <log_format>journaId</log_format>  
    <location>journaId</location>  
  </localfile>  
  
  <localfile>  
    <log_format>syslog</log_format>  
    <location>/var/ossec/logs/active-responses.log</location>  
  </localfile>  
  
  <localfile>  
    <log_format>syslog</log_format>  
    <location>/var/log/dpkg.log</location>  
  </localfile>  
  
  <localfile>  
    <log_format>json</log_format>  
    <location>/var/log/suricata/eve.json</location>  
  </localfile>
```

Figure 36 : Déclaration de `eve.json` dans la configuration Wazuh

Après le redémarrage de Wazuh Manager, `ossec.log` a indiqué que Logcollector analysait le fichier `/var/log/suricata/eve.json`. Ce message valide l'ouverture de la source par Wazuh. Il est recherché avant l'envoi du trafic afin d'éviter d'attribuer une absence d'alerte au capteur alors que le fichier n'est pas collecté.

```
sudo systemctl restart wazuh-manager  
sudo grep -i 'suricata/eve.json' /var/ossec/logs/ossec.log | tail
```

```

html. Some times were truncated, use -f to show in full.
socadmin@soc01:~$ sudo grep -iE 'suricata|eve\.json' /var/ossec/logs/ossec.log | tail -n 20
2026/08/18 21:23:45 wazuh-logcollector: INFO: (1950): Analyzing file: '/var/log/suricata/eve.
json'.
socadmin@soc01:~$ sudo grep -ic '"suricata"' /var/ossec/logs/alerts/alerts.json
3
socadmin@soc01:~$ sudo grep -i '"suricata"' /var/ossec/logs/alerts/alerts.json | tail -n 3 |
grep -o '"signature": "[^"]+"'
socadmin@soc01:~$ sudo grep -i '"suricata"' /var/ossec/logs/alerts/alerts.json | tail -n 1 |
python3 -m json.tool
{
  "timestamp": "2026-08-18T20:39:12.258+0000",
  "rule": {
    "level": 7,
    "description": "New dpkg (Debian Package) installed.",
    "id": "2902",
    "firedtimes": 63,
    "mail": false,
    "groups": [
      "syslog",
      "dpkg",
      "config_changed"
    ],
    "pci_dss": [
      "10.6.1",
      "10.2.7"
    ],
    "pgp13": [
      "4.10"
    ],
    "gdpr": [
      "IV_35.7.d"
    ],
    "hipaa": [
      "164.312.b"
    ],
    "nist_800_53": [
      "AU.6",
      "AU.14"
    ],
    "tsc": [
      "CC7.2",
      "CC7.3",
      "CC6.8",
      "CC8.1"
    ]
  },
  "agent": {
    "id": "000",
    "name": "soc01"
  },
  "manager": {
    "name": "soc01"
  },
  "id": "1787085552.119796",
  "full_log": "2026-08-18 20:39:12 status installed suricata:amd64 1:7.0.3-1build3",
  "decoder": {
    "name": "dpkg-decoder"
  },
  "data": {
    "dpkg_status": "status installed",
    "package": "suricata",
    "arch": "amd64",
    "version": "1:7.0.3-1build3"
  },
  "location": "/var/log/dpkg.log"
}
socadmin@soc01:~$

```

Figure 37 : Lecture de eve.json confirmée par Wazuh Logcollector

3.6.6 Exécution du test HTTP SID 2100498

Une requête HTTP contrôlée a été envoyée vers testmynids.org/uid/index.html. La réponse renvoie la chaîne uid=0(root) gid=0(root) groups=0(root), volontairement choisie pour correspondre à la signature GPL ATTACK_RESPONSE id check returned root. Le test ne lance aucune commande privilégiée sur soc01 ; il transmet seulement un contenu HTTP reconnaissable.

```
curl http://testmynids.org/uid/index.html
```

```

socadmin@soc01:~$ curl http://testmynids.org/uid/index.html
uid=0(root) gid=0(root) groups=0(root)
socadmin@soc01:~$

```

Figure 38 : Requête HTTP contrôlée utilisée pour le test IDS

La recherche immédiate dans fast.log a fait apparaître [1:2100498:7], la classification Potentially Bad Traffic, une priorité 2, le protocole TCP et l'adresse privée 10.0.2.4. Ces champs constituent la preuve directe que Suricata a inspecté et classé le trafic.

```
sudo grep '2100498' /var/log/suricata/fast.log | tail
```

```
socadmin@soc01:~$ sudo grep '2100498' /var/log/suricata/fast.log | tail -n 3
08/18/2026-21:32:47.555446  [**] [1:2100498:7] GPL ATTACK_RESPONSE id check returned root [**
[Classification: Potentially Bad Traffic] [Priority: 2] {TCP} 10.0.2.4
38270
socadmin@soc01:~$
```

Figure 39 : Déclenchement de la signature Suricata SID 2100498

Le choix d'une signature numérotée facilite le suivi entre les fichiers et le SIEM. Il évite de se contenter d'une augmentation globale du nombre d'événements et fournit un marqueur précis pour la recherche.

3.6.7 Validation de Filebeat et de l'indexeur

Filebeat assure le transport des alertes produites par Wazuh Manager vers Wazuh Indexer. La commande `systemctl status filebeat` a d'abord confirmé que le service est chargé, activé au démarrage et en exécution. Cette preuve est complétée par un test actif de sa destination.

```
sudo systemctl status filebeat --no-pager
```

```
socadmin@soc01:~$ sudo systemctl status filebeat --no-pager
● filebeat.service - Filebeat sends log files to Logstash or directly to Elasticsearch.
   Loaded: loaded (/usr/lib/systemd/system/filebeat.service; enabled; preset: enabled)
   Active: active (running) since Tue 2026-08-18 22:06:32 UTC; 21min ago
     Docs: https://www.elastic.co/products/beats/filebeat
    Main PID: 13997 (filebeat)
      Tasks: 7 (limit: 9502)
     Memory: 14.5M (peak: 15.7M)
        CPU: 232ms
      CGroup: /system.slice/filebeat.service
              └─13997 /usr/share/filebeat/bin/filebeat --environment systemd -c /etc/...

Aug 18 22:06:32 soc01 systemd[1]: Started filebeat.service - Filebeat sends log...rch..
Hint: Some lines were ellipsized, use -l to show in full.
socadmin@soc01:~$
```

Figure 40 : Service Filebeat actif après la configuration de la chaîne Wazuh

filebeat test output a ensuite contrôlé l'URL `https://127.0.0.1:9200`, la résolution, l'ouverture de la socket, la validation de la chaîne de certificats, le handshake TLS 1.2 et le dialogue avec le serveur. Toutes les étapes ont retourné OK, ce qui confirme que Filebeat peut joindre l'indexeur local.

```
sudo filebeat test output
```

```
socadmin@soc01:~$ sudo filebeat test output
elasticsearch: https://127.0.0.1:9200...
  parse url... OK
  connection...
    parse host... OK
    dns lookup... OK
  addresses: 127.0.0.1
  dial up... OK
  TLS...
    security: server's certificate chain verification is enabled
  handshake... OK
  TLS version: TLSv1.2
  dial up... OK
  talk to server... OK
  version: 7.10.2
socadmin@soc01:~$
```

Figure 41 : Test de sortie Filebeat vers le Wazuh Indexer avec TLS

3.6.8 Visualisation dans Wazuh Threat Hunting

La dernière vérification a été réalisée dans Wazuh Dashboard. Un filtre sur le groupe Suricata a fait apparaître l'événement dans Threat Hunting. Cette vue constitue le point d'exploitation pour l'analyste : elle permet de rechercher les documents, de consulter leurs champs et de les rapprocher d'autres événements de sécurité.

```
Filtre Threat Hunting : rule.groups:suricata
```

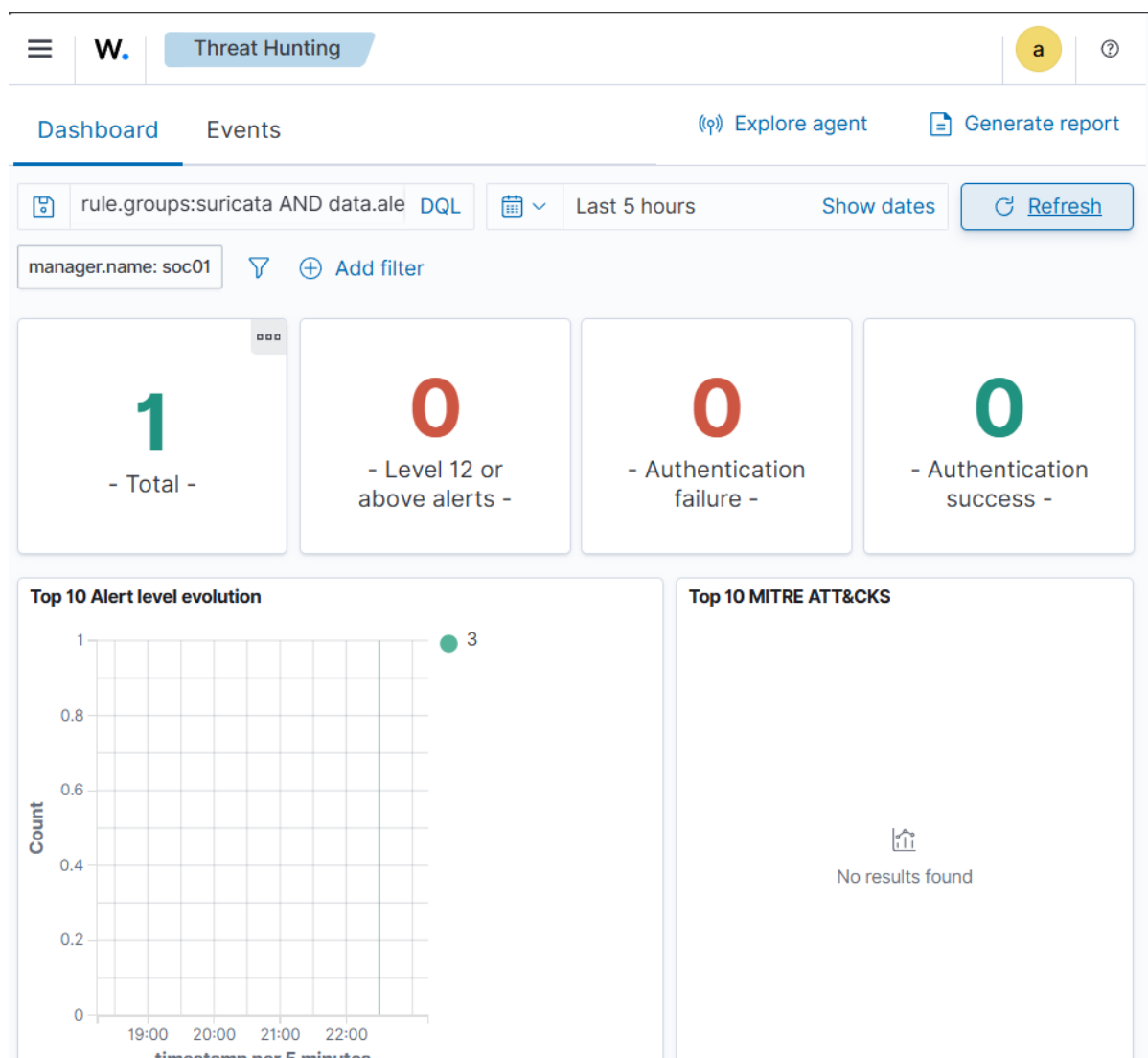


Figure 42 : Événement du groupe Suricata visible dans Wazuh Threat Hunting

Une rotation du mot de passe administrateur de l'indexeur a été effectuée pendant les contrôles. Le secret réel n'est pas reproduit dans le rapport. Après cette modification, Filebeat a été redémarré, le test de sortie a été relancé et un nouveau trafic HTTP a été généré. L'événement Suricata visible dans Threat Hunting confirme que la chaîne est restée fonctionnelle après la rotation.

```
sudo /usr/share/wazuh-indexer/plugins/opensearch-security/tools/wazuh-passwords-tool.sh \
-u admin -p '<mot_de_passe_fort_non_documente>'
sudo systemctl restart filebeat
sudo filebeat test output
```

Les contrôles réunis couvrent ainsi les différents niveaux de la plateforme : présence de la règle, détection du SID dans fast.log, production de eve.json, lecture par Wazuh Logcollector, transport TLS par Filebeat et affichage d'un événement du groupe Suricata dans Threat Hunting.

Tableau 15 : Problèmes, analyses et solutions de la Phase 3

Problème rencontré	Analyse	Solution appliquée
Fichier de règles absent	Suricata était installé sans jeu de règles exploitable	Exécution de suricata-update puis nouveau test suricata -T
Risque d'écouter la mauvaise interface	Un service actif ne garantit pas l'observation du trafic utile	Vérification explicite de eth0 et du mode AF_PACKET
Événement non exploitable comme JSON	Une lecture texte limite le décodage des champs	Déclaration localfile avec log_format json

Problème rencontré	Analyse	Solution appliquée
Doute sur l'accès à l'indexeur	La détection peut réussir alors que le transport échoue	Contrôle avec filebeat test output et TLS
Rotation du secret administrateur	Les composants doivent conserver des identifiants cohérents	Redémarrage de Filebeat puis revalidation du transport et du trafic
Recherche difficile dans le Dashboard	Le volume d'événements masque le scénario	Filtre rule.groups:suricata dans Threat Hunting

3.7 Phase 4 — Exploitation SIEM et tableaux de bord

Après la validation technique de Suricata et de la collecte EVE JSON, la quatrième phase a porté sur l'usage opérationnel du SIEM. L'objectif n'était plus seulement de voir une alerte, mais de pouvoir la retrouver rapidement, afficher les champs utiles, sauvegarder une recherche et construire des vues réutilisables pour les sources ajoutées par la suite.

Architecture finale du laboratoire SOC Azure

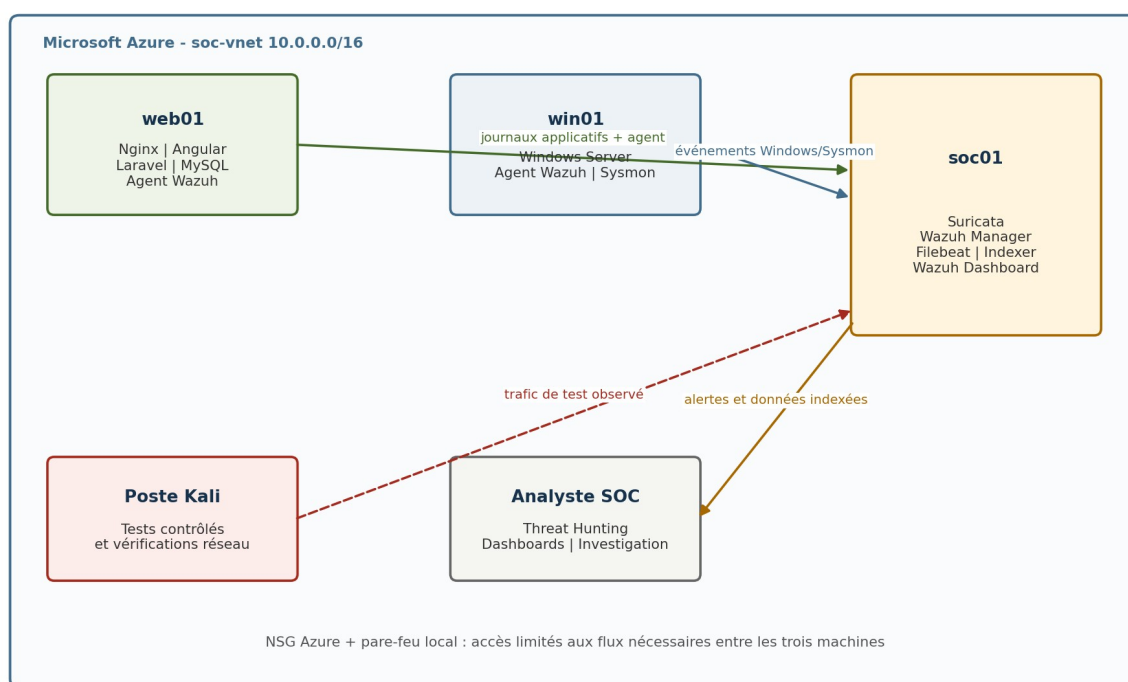


Figure 43 : Architecture finale du laboratoire SOC sur Microsoft Azure

Tableau 18 : Inventaire final des machines et rôles de l'architecture

Machine	Système	Rôle	Sources principales
soc01	Ubuntu Server	SOC central et capteur réseau	Suricata, Wazuh, Filebeat
win01	Windows	Endpoint supervisé	Security, System, Application, Sysmon
web01	Ubuntu Server	Hébergement de SecureVault	Nginx, Laravel, security.log
Kali	Linux	Poste de validation	trafic contrôlé dans le périmètre autorisé

3.7.1 Vérification du transport Filebeat-Indexer

Le premier contrôle a porté sur la sortie de Filebeat. Une détection locale peut réussir alors que l'Indexer ne reçoit rien ; dans ce cas, le Dashboard reste vide malgré un capteur fonctionnel. La commande filebeat test

output vérifie la résolution de la destination, l'établissement de la session TLS, l'authentification et la capacité à dialoguer avec le service OpenSearch/Wazuh Indexer.

```
sudo systemctl status filebeat --no-pager
sudo filebeat test output
sudo journalctl -u filebeat -n 50 --no-pager
```

Le premier test a retourné une erreur d'authentification après la rotation du mot de passe administrateur. Le problème ne venait ni du réseau ni du certificat : les identifiants conservés par Filebeat n'étaient plus synchronisés avec ceux de l'Indexer. Le secret a été mis à jour dans le keystore, puis le service a été redémarré. Le nouveau test a confirmé la connexion TLS et la communication avec 127.0.0.1:9200.

```
echo '<utilisateur_indexer>' | sudo filebeat keystore add username --stdin --force
echo '<mot_de_passe_masque>' | sudo filebeat keystore add password --stdin --force
sudo systemctl restart filebeat
sudo filebeat test output
```

Les valeurs sensibles ne sont pas reproduites dans le rapport. L'intérêt de cet incident est le diagnostic : le handshake TLS réussi suivi d'un refus d'authentification oriente vers le secret, alors qu'une absence de connexion aurait orienté vers le port, le certificat ou le service.

3.7.2 Méthode de recherche dans Threat Hunting

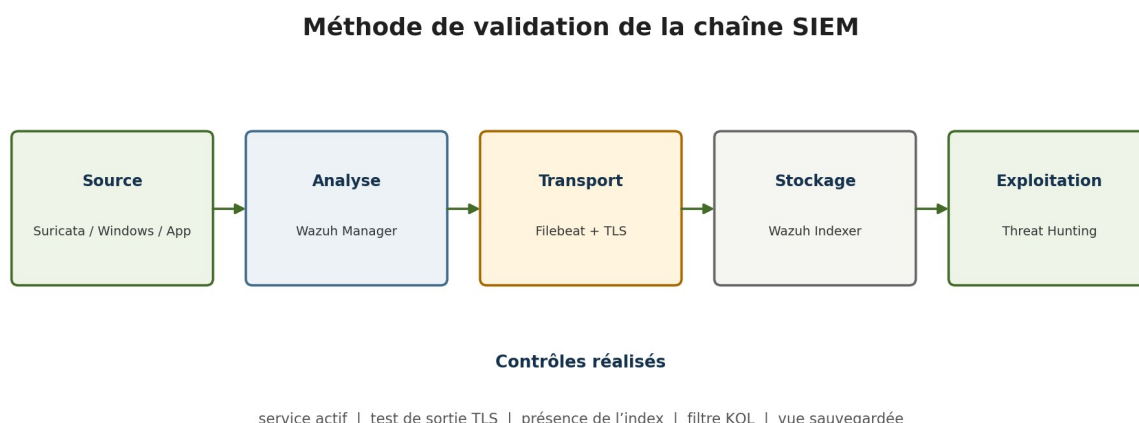


Figure 44 : Démarche de validation de la Phase 4 dans Wazuh

La recherche a été organisée autour de filtres simples et explicables. Le filtre `rule.groups:suricata` isole les alertes provenant du NIDS. Pour Windows, `agent.name:win01` identifie la machine, puis un champ tel que `data.win.eventdata.destinationPort` permet de réduire le résultat. Pour SecureVault, les labels de source et le nom de l'événement applicatif servent de point d'entrée.

Construction d'une recherche Threat Hunting



Exemples de filtres sauvegardés

```
rule.groups:suricata | agent.name:win01 | data.win.eventdata.destinationPort:5985
```

Figure 45 : Construction d'une recherche Threat Hunting appliquée aux événements Suricata

Tableau 19 : Requêtes et vues utilisées pendant l'investigation SIEM

Vue	Filtre de départ	Utilité
Suricata	rule.groups:suricata	retrouver les alertes réseau et lire signature, source et destination
Windows	agent.name:win01	isoler l'endpoint avant d'analyser EventID, processus et ports
Sysmon réseau	data.win.eventdata.destinationPort:5985	étudier une connexion précise sans conclure trop vite
SecureVault	app.name:securevault	regrouper les actions métier et les échecs de sécurité
Nginx	log.file.path:/var/log/nginx/access.log	relier la requête HTTP au contexte applicatif

3.7.3 Ajout de labels et normalisation minimale

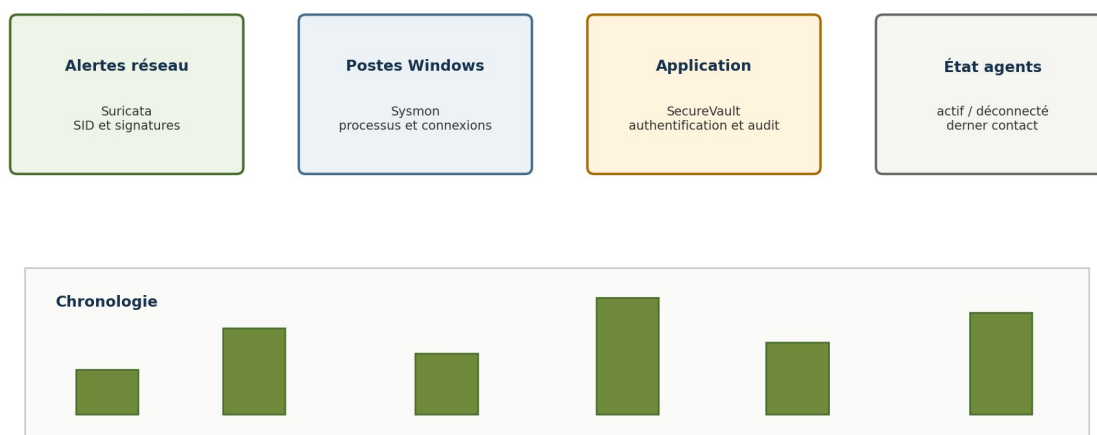
Lorsque plusieurs fichiers sont collectés, leur chemin ne suffit pas toujours à construire une vue lisible. Des labels ont donc été ajoutés aux blocs localfile afin d'identifier l'application, le type de serveur et l'environnement. Cette normalisation reste volontairement légère : elle n'altère pas les champs d'origine et fournit des clés stables pour les filtres.

```
<localfile>
<location>/var/log/suricata/eve.json</location>
<log_format>json</log_format>
<label key="source.type">network_ids</label>
<label key="environment">azure_lab</label>
</localfile>
```

3.7.4 Construction des tableaux de bord

Les tableaux de bord ont été conçus comme des points d'entrée vers l'investigation. Ils présentent le volume d'événements, les agents concernés, les groupes de règles, les ports observés et l'évolution dans le temps. Un clic ou un filtre doit permettre de revenir aux documents bruts ; le graphique ne remplace pas la lecture de l'événement.

Organisation du tableau de bord de supervision



Les tableaux de bord accélèrent le tri ; l'analyse détaillée reste effectuée dans Threat Hunting.

Figure 46 : Organisation du tableau de bord de supervision des événements de sécurité

La phase a abouti à des recherches sauvegardées et à une organisation stable des vues. Cette base a ensuite été réutilisée pour Windows Monitoring et pour les événements de SecureVault, ce qui évite de reconstruire la méthode d'analyse à chaque nouvelle source.

3.8 Phase 5 — Supervision Windows et corrélation Sysmon

La cinquième phase ajoute une source endpoint Windows. Ce choix est important dans un SOC : un capteur réseau décrit les communications, mais il ne donne pas toujours le processus à l'origine du flux, l'utilisateur connecté ou le contexte local. Sysmon complète cette visibilité en produisant des événements détaillés que l'agent Wazuh transmet au Manager.

3.8.1 Création de win01 dans Azure

win01 a été déployée dans le même réseau virtuel que soc01 afin d'utiliser les adresses privées pour la communication avec Wazuh. L'adresse privée observée est 10.0.0.4. L'accès RDP a été limité par le NSG à la source d'administration ; les ports agents 1514/TCP et 1515/TCP sont destinés respectivement à la transmission des événements et à l'enrôlement.

Intégration de win01 dans le réseau Azure



La machine Windows apporte une source endpoint différente du serveur Linux et du capteur réseau.

Objectif : corrélér les processus, connexions et événements système avec les autres alertes du SOC.

Figure 47 : Intégration de la machine Windows win01 au réseau Azure

3.8.2 Installation et enregistrement de l'agent Wazuh

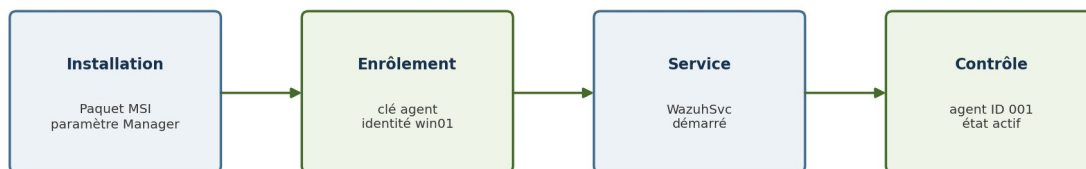
L'agent a été installé depuis une console PowerShell exécutée avec des droits administrateur. Le nom win01 a été fixé lors de l'installation afin de conserver une identité lisible dans le Dashboard. Le service Wazuh a ensuite été démarré et contrôlé localement, puis la liste des agents sur soc01 a confirmé l'enregistrement sous l'identifiant 001.

```

$msi = "$env:TEMP\wazuh-agent.msi"
Invoke-WebRequest -Uri "https://packages.wazuh.com/4.x/windows/wazuh-agent-4.14.4-1.msi" -OutFile $msi
Start-Process msixec.exe -Wait -ArgumentList "/i \"$msi\" /q WAZUH_MANAGER=10.0.2.4 WAZUH_AGENT_NAME=win01"
Start-Service WazuhSvc
Get-Service WazuhSvc

# Contrôle sur soc01
sudo /var/ossec/bin/agent_control -lc
  
```

Enregistrement et supervision de l'agent Windows



Le statut actif confirme la communication ; il ne prouve pas encore que les canaux utiles sont collectés.

Figure 48 : Chaîne d'enregistrement et de supervision de l'agent Windows

3.8.3 Installation et collecte de Sysmon

Sysmon a été installé avec une configuration XML afin de sélectionner les catégories d'événements utiles. Après installation, le canal Microsoft-Windows-Sysmon/Operational a été vérifié dans l'Event Viewer. L'agent Wazuh a ensuite été configuré pour lire ce canal au format eventchannel.

```
.\Sysmon64.exe -accepteula -i .\sysmonconfig.xml
Get-Service Sysmon64
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 5

<!-- C:\Program Files (x86)\ossec-agent\ossec.conf -->
<localfile>
  <location>Microsoft-Windows-Sysmon/Operational</location>
  <log_format>eventchannel</log_format>
</localfile>
```

La modification a été appliquée par un redémarrage du service WazuhSvc. Le contrôle a porté à la fois sur la présence des événements dans Windows et sur leur arrivée dans l'index Wazuh ; vérifier un seul côté aurait laissé un doute sur la transmission.

Sources Windows collectées sur win01

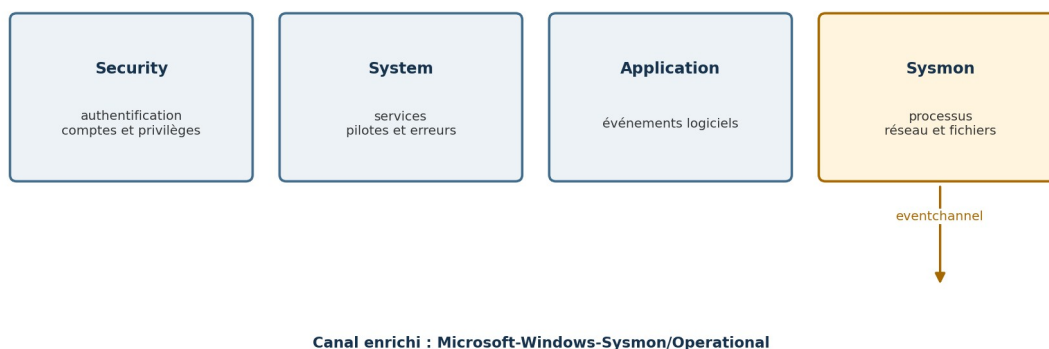


Figure 49 : Sources Windows et canal Microsoft-Windows-Sysmon/Operational

Tableau 20 : Sources de journaux collectées sur win01

Source	Exemples d'informations	Intérêt SOC
Security	ouvertures de session, comptes et privilèges	identifier l'identité et le type de connexion
System	services, pilotes et erreurs	comprendre les changements de l'OS
Application	événements des logiciels	relier un dysfonctionnement à un composant
Sysmon	processus, réseau, fichiers et GUID	enrichir la chronologie technique

3.8.4 Analyse d'un événement réseau Sysmon

Un événement réseau indexé depuis win01 contient notamment le protocole TCP, le port destination 5985, l'adresse ::1, le processus System ou svchost.exe, un port source dynamique, l'indicateur initiated et un ProcessGuid. Le port 5985 correspond couramment à WinRM en HTTP et ::1 désigne la boucle locale IPv6. Ces valeurs décrivent le contexte ; elles ne prouvent pas une attaque.

Lecture d'un événement réseau Sysmon indexé

Agent	Protocole	Port destination	Destination
win01 (ID 001)	tcp	5985	::1
Image	Initiated	Source port	Process GUID
System / svchost.exe	true / false observé	valeur dynamique	présent

Interprétation : événement de connexion réseau riche en contexte, utile à l'investigation ; la présence du port 5985 ne suffit pas, seule, à qualifier l'événement de malveillant.

Figure 50 : Champs observés dans un événement réseau Sysmon indexé

Tableau 21 : Champs utiles d'un événement réseau Sysmon

Champ	Valeur observée	Lecture
destinationPort	5985	port couramment associé à WinRM HTTP
destinationIp	::1	boucle locale IPv6
protocol	tcp	transport de la connexion
image	System / svchost.exe	contexte du processus système
initiated	true ou false selon l'événement	sens de l'initiation indiqué par Sysmon
processGuid	GUID présent	identifiant utile pour relier plusieurs événements

Cette interprétation prudente est essentielle devant un jury comme dans un SOC réel : un port connu ou un processus système ne doit pas être qualifié automatiquement de malveillant. La décision nécessite la fréquence, l'utilisateur, la destination, l'horaire et les événements voisins.

3.8.5 Tableau de bord Windows Monitoring

Une vue Windows Monitoring a été créée afin de regrouper l'agent, les événements Sysmon, les processus et les ports observés. Le tableau de bord facilite le repérage d'un pic ou d'une valeur inhabituelle, puis l'analyste revient dans Discover ou Threat Hunting pour examiner les champs du document complet.

Vue logique du tableau de bord Windows Monitoring

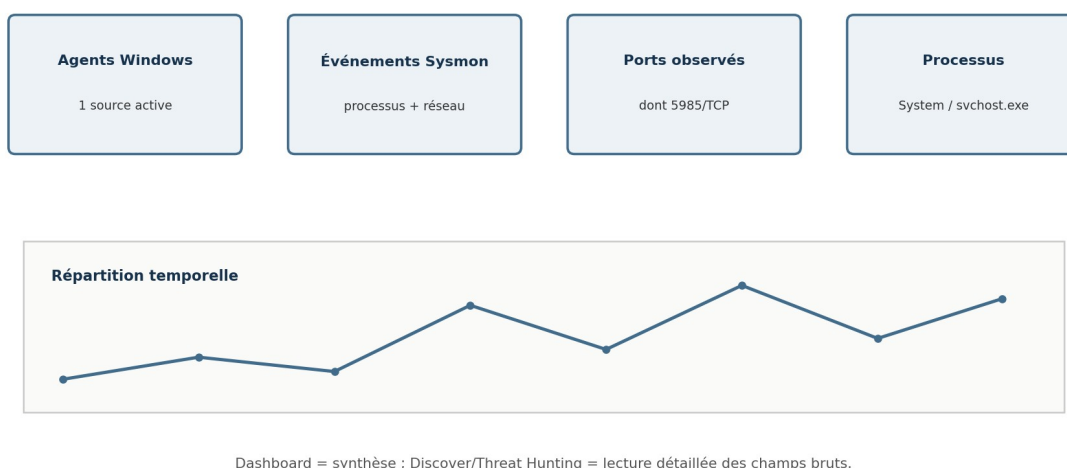


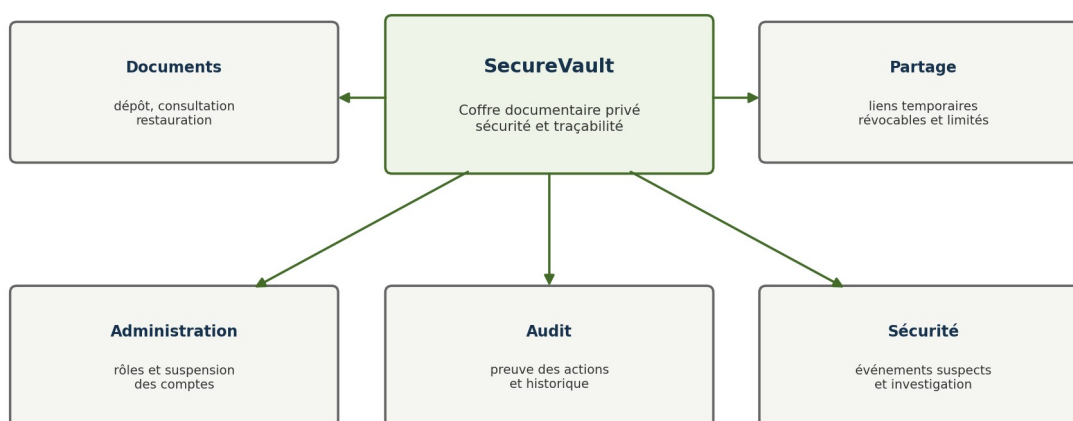
Figure 51 : Organisation du tableau de bord Windows Monitoring

La Phase 5 confirme que le SOC ne dépend plus d'une seule source Linux/réseau. win01 est visible comme agent actif, les événements EventChannel et Sysmon sont indexés, et une vue spécifique permet leur exploitation.

3.9 Phase 6 — Déploiement et supervision de SecureVault

La sixième phase relie le SOC à un actif métier concret. SecureVault est un coffre documentaire privé inspiré d'un service de stockage en ligne, mais centré sur le contrôle des partages et la traçabilité. L'application permet d'organiser des fichiers, de créer des liens temporaires et révocables, de limiter les téléchargements, de restaurer ou supprimer des documents et d'administrer les comptes.

SecureVault : valeur métier et surface à protéger



Données sensibles : comptes, documents, liens de partage, rôles, adresses IP et traces d'accès.

Figure 52 : Périmètre fonctionnel et données sensibles de SecureVault

Tableau 22 : Fonctions SecureVault et événements de sécurité associés

Fonction	Donnée sensible	Événement à tracer
Authentification	compte et session	succès, échec, blocage et déconnexion
Documents	nom, propriétaire et fichier	upload, download, suppression et restauration
Partage	jeton, expiration et quota	création, utilisation, révocation et dépassement
Administration	rôle et état du compte	changement de rôle et suspension
Audit	auteur, IP, objet et résultat	consultation et examen d'un incident

3.9.1 Déploiement de web01

Une VM Ubuntu sans interface graphique a été créée pour héberger l'application. La séparation entre web01 et soc01 limite l'impact d'une anomalie applicative sur les services centraux du SOC. Le NSG de web01 autorise seulement l'administration depuis la source prévue, l'accès web nécessaire à la démonstration et les communications privées de l'agent vers soc01.

Déploiement réseau de web01



Le service web reste séparé du serveur SOC ; seuls les flux nécessaires sont autorisés.

Cette séparation évite d'héberger l'actif métier et les composants centraux de supervision sur la même VM.

Figure 53 : Topologie de déploiement de web01 et filtrage réseau associé

Pendant la démonstration, l'accès a été volontairement limité à la source d'administration. Le fait que l'application soit accessible depuis le poste autorisé mais pas depuis un téléphone utilisant une autre adresse publique confirme l'effet du filtrage par source, plutôt qu'un défaut de l'application.

3.9.2 Installation de la pile web

```

sudo apt update
sudo apt install -y nginx mysql-server php-fpm php-cli php-mysql php-xml php-curl php-mbstring php-zip
unzip git curl
sudo systemctl enable --now nginx mysql
php -v && nginx -v && mysql --version
  
```

Nginx reçoit les requêtes HTTP, sert la version compilée d'Angular et transmet les appels API à Laravel via PHP-FPM. MySQL conserve les comptes, les métadonnées documentaires, les dossiers, les liens de partage et l'historique. Les fichiers déposés restent dans l'espace de stockage de l'application avec des permissions limitées au compte du service web.

Modules fonctionnels de SecureVault

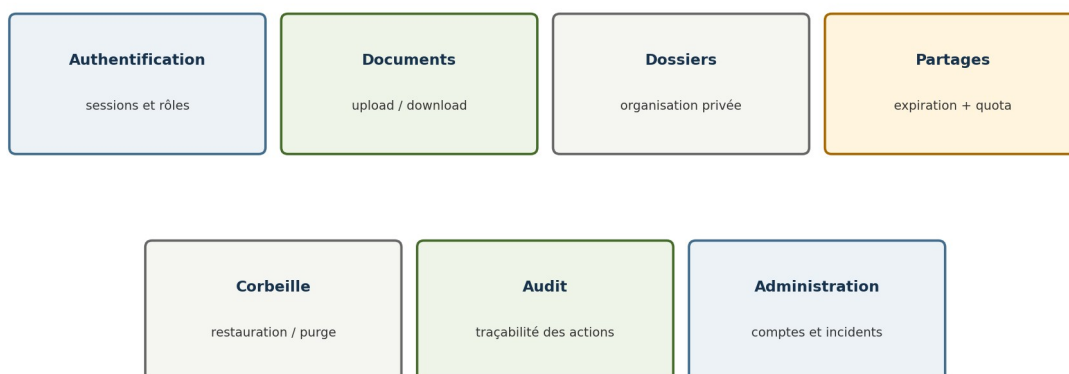


Figure 54 : Principaux modules fonctionnels de l'application SecureVault

Pile applicative déployée sur web01

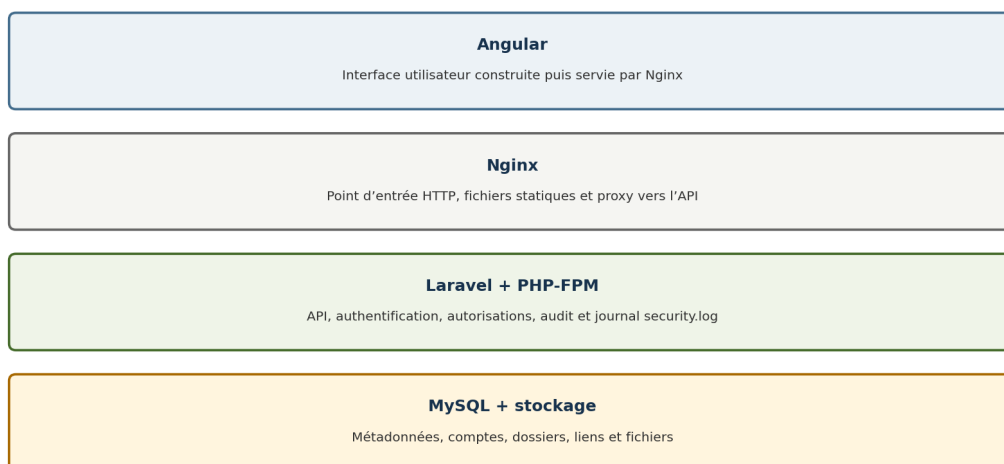


Figure 55 : Architecture applicative Nginx, Angular, Laravel et MySQL

Tableau 23 : Composants de la pile applicative déployée sur web01

Composant	Rôle	Contrôle appliqué
Nginx	point d'entrée et journaux HTTP	server block, test nginx -t et accès limité
Angular	interface utilisateur	build de production et routes SPA
Laravel	API, autorisation et audit	APP_ENV, clé, cache et permissions
PHP-FPM	exécution de l'API	socket local et service systemd
MySQL	données structurées	compte applicatif dédié et migration
Wazuh Agent	collecte du serveur web	service actif et liaison privée vers soc01

3.9.3 Préparation de Laravel et Angular

Le code a été transféré sur web01, puis les dépendances ont été installées dans le répertoire du projet. Le fichier .env contient uniquement les paramètres de l'environnement de déploiement et n'est pas reproduit dans le rapport. Les migrations ont préparé la base et les caches Laravel ont été régénérés. La partie Angular a ensuite été compilée pour produire les fichiers statiques servis par Nginx.

```
cd /var/www/securevault/backend
composer install --no-dev --optimize-autoloader
cp .env.example .env
php artisan key:generate
php artisan migrate --force
php artisan storage:link
php artisan config:cache && php artisan route:cache

cd /var/www/securevault/frontend
npm ci
npm run build

sudo chown -R www-data:www-data /var/www/securevault/backend/storage
/var/www/securevault/backend/bootstrap/cache
sudo find /var/www/securevault/backend/storage -type d -exec chmod 750 {} \;
sudo nginx -t
sudo systemctl reload nginx
```

3.9.4 Conception du journal security.log

Les journaux techniques Laravel ne suffisent pas à expliquer une action métier. Un canal dédié écrit donc des événements structurés dans security.log. Chaque ligne contient un type d'événement, une gravité, l'utilisateur lorsqu'il est connu, l'adresse source, la route, le résultat et un identifiant de requête. Les mots de passe, jetons de session et contenus documentaires sont exclus.

Structure du journal de sécurité SecureVault

```
{
  "timestamp":      "2026-08-28T18:42:31Z",
  "event":          "login_failed",
  "severity":       "warning",
  "user_id":        42,
  "ip":             "source_redacted",
  "route":          "/api/login",
  "status":         401,
  "request_id":     "uuid"
}
```

Pourquoi ce format ?
- contexte d'investigation
- corrélation par utilisateur/IP
- décodage JSON dans Wazuh

Figure 56 : Structure d'un événement de sécurité écrit dans security.log

Tableau 24 : Structure d'un événement applicatif SecureVault

Champ	Rôle	Exemple non sensible
timestamp	ordonner les événements	2026-08-28T18:42:31Z
event	nom stable de l'action	login_failed ou share_revoked
severity	prioriser l'analyse	info, warning ou high

Champ	Rôle	Exemple non sensible
user_id	relier l'action au compte	identifiant interne
ip	identifier la source	adresse masquée dans le rapport
route / object_id	préciser la cible	/api/login ou identifiant du document
request_id	corréler Nginx et Laravel	UUID

3.9.5 Installation de l'agent et collecte des journaux

L'agent Wazuh de web01 a été configuré avec l'adresse privée de soc01. Les premiers contrôles ont été effectués avec systemctl et journalctl, car un agent installé mais arrêté ne transmet aucune donnée. Après validation du service, les blocs localfile ont été ajoutés pour security.log, laravel.log, access.log et error.log.

```

sudo systemctl status wazuh-agent --no-pager
sudo journalctl -u wazuh-agent -n 80 --no-pager
sudo tail -n 5 /var/www/securevault/backend/storage/logs/security.log

<localfile>
<location>/var/www/securevault/backend/storage/logs/security.log</location>
<log_format>json</log_format>
<label key="app.name">securevault</label>
<label key="source.type">application_security</label>
</localfile>
<localfile>
<location>/var/log/nginx/access.log</location>
<log_format>apache</log_format>
</localfile>
<localfile>
<location>/var/log/nginx/error.log</location>
<log_format>syslog</log_format>
</localfile>

```

Sources collectées par l'agent Wazuh sur web01

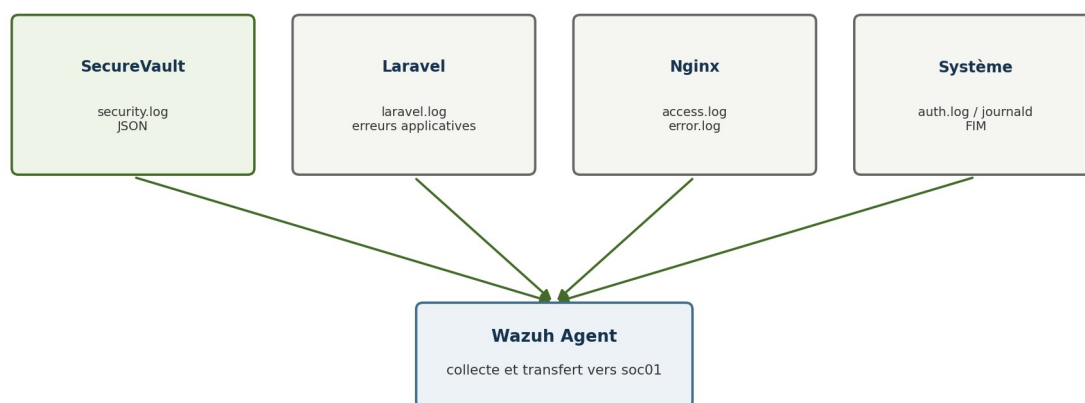


Figure 57 : Sources SecureVault et Nginx déclarées dans l'agent Wazuh

3.9.6 Règles Wazuh adaptées à SecureVault

Des règles locales ont été définies dans un fichier séparé afin d'éviter de modifier le ruleset fourni par Wazuh. Elles s'appuient sur les noms d'événements stables produits par l'application. Une règle simple identifie l'action ; une règle de fréquence peut ensuite augmenter le niveau lorsque plusieurs échecs de connexion proviennent de la même source dans un intervalle court.

```
<group name="securevault,application,">
  <rule id="100500" level="5">
    <decoded_as>json</decoded_as>
    <field name="event">login_failed</field>
    <description>SecureVault : échec d'authentification</description>
    <group>authentication_failed,</group>
  </rule>
  <rule id="100501" level="10" frequency="5" timeframe="120">
    <if_matched_sid>100500</if_matched_sid>
    <same_field>ip</same_field>
    <description>SecureVault : échecs répétés depuis la même source</description>
    <mitre><id>T1110</id></mitre>
  </rule>
</group>
```

Les règles ont été validées avec wazuh-logtest à partir d'un exemple de journal expurgé. Ce contrôle vérifie le décodage et la règle sans devoir provoquer une attaque contre l'application.

```
sudo /var/ossec/bin/wazuh-logtest
sudo systemctl restart wazuh-manager
```

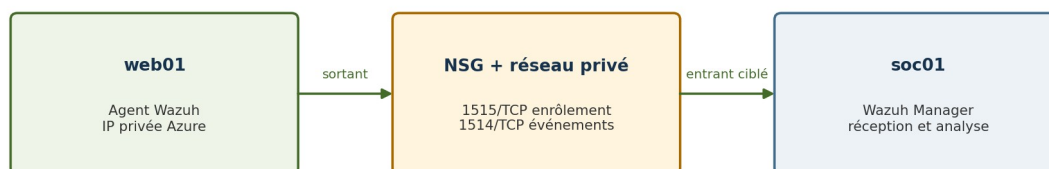
Tableau 25 : Règles Wazuh adaptées aux événements SecureVault

Événement	Niveau indicatif	Interprétation
login_failed	5	échec unique à contextualiser
échecs répétés même IP	10	comportement compatible avec une tentative systématique
share_created / share_revoked	3 à 5	action d'audit, sensible selon le document
mass_download_detected	10	volume inhabituel nécessitant une investigation
role_changed / account_suspended	7 à 9	action administrative à haute valeur de traçabilité

3.9.7 Validation réseau entre web01 et soc01

La communication agent a été vérifiée au niveau Azure avec Network Watcher / test IP flow. Les résultats Allow pour les règles d' enrôlement et de transmission confirment que le NSG n'interrompt pas les ports attendus. Cette vérification est complémentaire à systemctl : un service actif localement peut rester isolé par une règle réseau.

Flux autorisés entre web01 et soc01



Le contrôle Network Watcher / test IP flow confirme que la règle attendue autorise le flux agent.

Figure 58 : Flux autorisés entre web01 et soc01

3.9.8 Investigation des événements applicatifs

Les événements SecureVault sont recherchés à partir du label app.name, puis analysés avec l'utilisateur, l'adresse source, la route et le résultat. Le request_id permet de rapprocher une action Laravel de la requête correspondante dans access.log. Cette corrélation donne davantage de valeur qu'un message générique, car elle relie le contexte web et le contexte métier.



Figure 59 : Investigation d'un événement SecureVault dans Wazuh

La validation finale de la phase repose sur des actions normales et contrôlées de l'application, notamment l'authentification, la gestion des documents et les opérations administratives. La campagne SQLi/XSS/CSRF et force brute envisagée au départ n'a pas été retenue dans le périmètre final du rapport. Aucun résultat offensif n'est donc inventé ; les règles décrivent la capacité d'observation disponible.

3.10 Validation de bout en bout et lecture MITRE ATT&CK

La validation globale consiste à suivre une action depuis sa source jusqu'au Dashboard. Pour chaque source, cinq preuves sont recherchées : l'action ou le trafic, la ligne de journal, la lecture par Wazuh, l'application d'une règle et la présence du document indexé. Cette méthode évite de conclure à partir d'une seule capture.

Chemin complet d'un événement de sécurité



Preuve attendue à chaque étape

requête ou action → ligne de journal → réception → règle déclenchée → document indexé et lisible

Cette continuité permet de diagnostiquer une perte de visibilité sans attribuer l'erreur au mauvais composant.

Figure 60 : Chemin complet d'un événement, de SecureVault au Dashboard

3.10.1 Interprétation avec MITRE ATT&CK

MITRE ATT&CK fournit un vocabulaire pour relier un comportement à une technique. Le mapping doit rester conditionnel : un échec de connexion isolé n'est pas du brute force et une connexion au port 5985 n'est pas une preuve d'utilisation malveillante de WinRM. Le tableau suivant distingue la donnée observée de la condition nécessaire à une qualification.

Tableau 26 : Correspondance entre scénarios observables et MITRE ATT&CK

Scénario	Source	Technique	Condition d'interprétation
Échecs d'authentification répétés	SecureVault / Windows	T1110 - Brute Force	répétition systématique dans une fenêtre courte
Abus d'un compte authentifié	SecureVault / Security	T1078 - Valid Accounts	action anormale réalisée avec un compte valide
Exploitation d'une interface exposée	Suricata / Nginx	T1190 - Exploit Public-Facing Application	signature ou preuve d'exploitation, pas une simple erreur HTTP
Découverte de services réseau	Suricata	T1046 - Network Service Scanning	connexions vers plusieurs ports ou hôtes
Modification de rôle ou de compte	SecureVault	T1098 - Account Manipulation	changement non autorisé ou inattendu
Collecte inhabituelle de documents	SecureVault	T1005 - Data from Local System	volume et contexte compatibles avec une collecte

3.11 Synthèse des résultats

Tableau 27 : Résultats finaux des six phases

Phase	Résultat principal	Preuve ou mesure
1	Infrastructure Azure et Wazuh opérationnelle	soc01, services systemd et Dashboard
2	Serveur durci par plusieurs couches	écarts CIS réduits de 106 à 5
3	Chaîne Suricata–Wazuh validée	SID 2100498, eve.json, Filebeat TLS et Threat Hunting

Phase	Résultat principal	Preuve ou mesure
4	Recherches et tableaux de bord structurés	filtres sauvegardés, labels et vues par source
5	win01 supervisée avec Sysmon	agent 001 actif et événement réseau indexé
6	SecureVault déployée et instrumentée	web01, security.log, Nginx et agent Wazuh

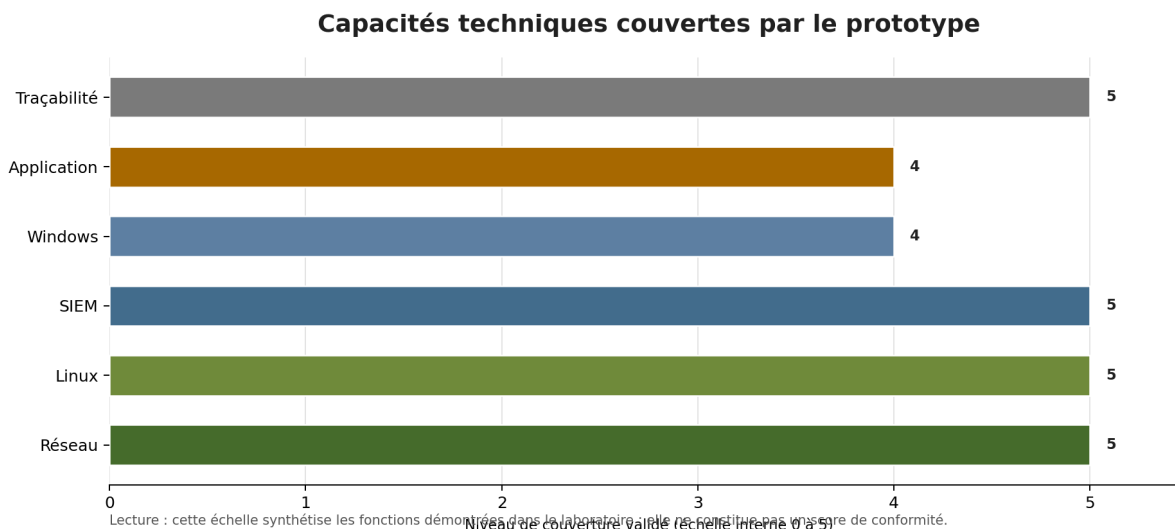


Figure 61 : Répartition des capacités de détection validées par source

Les mesures les plus significatives sont la réduction des écarts CIS, le chargement d'environ 52 360 règles Suricata, le déclenchement du SID 2100498, la réussite du test TLS Filebeat et l'indexation d'une source Windows puis applicative. Le graphique synthétise la couverture démontrée ; il ne représente ni un taux de conformité ni un niveau de maturité SOC certifié.

3.12 Difficultés rencontrées et solutions

Tableau 28 : Difficultés, causes et solutions mises en œuvre

Difficulté	Cause identifiée	Solution et enseignement
Partition racine saturée	cache temporaire vd_updater proche de 16 Go	arrêt du service, nettoyage ciblé et contrôle avant/après
Clé SSH refusée sous Windows	droits locaux trop larges sur le fichier PEM	restriction des ACL puis nouvelle connexion
Règles Suricata absentes	rules-file non disponible	suricata-update puis validation avec suricata -T
Filebeat non authentifié	secret incohérent après rotation du mot de passe	mise à jour du keystore et nouveau test TLS
Événement difficile à retrouver	volume de documents et filtre trop large	filtre par groupe, agent, champ et fenêtre temporelle
Agent web non communicant	service ou flux agent à vérifier	journalctl, contrôle du service et test IP flow Azure
Interprétation Sysmon ambiguë	champ technique isolé de son contexte	lecture conjointe du port, IP, processus, GUID et chronologie

3.13 Compétences mobilisées

La réalisation a mobilisé des compétences d'infrastructure, d'administration, de détection, de développement sécurisé et d'analyse. Le travail a surtout renforcé une méthode de diagnostic : partir du résultat attendu, isoler chaque dépendance, lire les journaux et conserver une preuve interprétable.

- Concevoir une topologie Azure avec VNet, sous-réseaux, NSG et plusieurs VM.
- Administrer et durcir Ubuntu avec systemd, UFW, SSH, Fail2ban, permissions et USG/CIS.

- Configurer Suricata, ses règles et le format EVE JSON.
- Exploiter Wazuh Manager, Filebeat, Indexer, Threat Hunting et les tableaux de bord.
- Déployer un agent Windows, installer Sysmon et interpréter les champs EventChannel.
- Déployer une application Laravel/Angular/MySQL derrière Nginx et l'instrumenter pour le SOC.
- Écrire et tester des règles Wazuh locales sans modifier le ruleset fourni.
- Présenter une analyse MITRE ATT&CK sans confondre donnée observable et preuve d'attaque.

3.14 Conclusion du chapitre

Les six phases ont construit une chaîne de supervision complète à l'échelle du laboratoire. Le socle Azure et Wazuh est protégé, Suricata apporte la visibilité réseau, les vues SIEM facilitent l'investigation, Sysmon enrichit la télémétrie endpoint et SecureVault fournit le contexte métier. Les résultats sont reliés à des commandes, des journaux et des événements observables.

Le principal apport n'est donc pas l'installation d'un Dashboard, mais la capacité à expliquer le chemin d'une donnée et à localiser une rupture. Cette démarche rend l'architecture compréhensible par le jury et réutilisable pour l'ajout d'autres agents ou d'autres applications.

Conclusion générale

L'objectif de ce projet était de concevoir et de mettre en œuvre un SOC Cloud capable de superviser des sources réseau, système, endpoint et applicatives dans Microsoft Azure. La réalisation a abouti à une architecture composée de trois machines principales : soc01 pour les composants centraux et Suricata, win01 pour la télémétrie Windows/Sysmon et web01 pour l'application SecureVault.

La première phase a établi le réseau Azure et la pile Wazuh. Elle a également montré l'importance de l'exploitation quotidienne du SOC lorsqu'une saturation de la partition racine a perturbé les services. La deuxième phase a renforcé le serveur par le filtrage NSG et UFW, une configuration SSH restrictive, Fail2ban, des options de montage adaptées et un cycle d'audit/remédiation CIS. Le nombre d'écarts relevés par USG est passé de 106 à 5, sans présenter ce résultat comme une conformité totale.

La troisième phase a validé la chaîne réseau : Suricata a chargé les règles Emerging Threats et a déclenché le SID 2100498 sur un trafic HTTP contrôlé. Wazuh a lu eve.json, Filebeat a transmis les alertes avec TLS et l'événement est devenu consultable dans Threat Hunting. La quatrième phase a transformé cette preuve technique en méthode d'exploitation grâce à des filtres, des labels, des recherches sauvegardées et des tableaux de bord.

La cinquième phase a ajouté win01 et Sysmon. L'événement réseau étudié montre l'intérêt d'un contexte endpoint : port, adresse, processus, direction et GUID peuvent être rapprochés au lieu de se limiter à un flux réseau. La sixième phase a enfin déployé SecureVault et centralisé ses journaux de sécurité, Laravel et Nginx. Le SOC est ainsi relié à des actions métier telles que l'authentification, le partage, la gestion de documents et l'administration des comptes.

Le projet répond donc à la problématique initiale : une chaîne multi-source a été construite, sécurisée et vérifiée de la production du journal jusqu'à l'investigation. La principale leçon est méthodologique. Un SOC fiable ne se résume pas à un écran d'alertes ; il exige une compréhension précise de chaque maillon et une interprétation prudente des données. Un port connu, un échec isolé ou un processus système n'est pas automatiquement malveillant.

Le prototype reste adapté à un laboratoire : le déploiement Wazuh central est all-in-one, cinq écarts CIS restent documentés et la campagne offensive applicative a été exclue du périmètre final. Pour une industrialisation, les perspectives prioritaires seraient la séparation des rôles Wazuh, une politique de rétention et de sauvegarde, la supervision automatique du stockage, un nom DNS avec HTTPS, des notifications d'incident et des procédures de réponse formalisées. Ces évolutions prolongent le travail réalisé sans remettre en cause la démonstration du prototype.

Bibliographie et webographie

Les références ont été consultées entre le 20 et le 31 août 2026. Les documentations officielles des éditeurs et des projets open source ont été privilégiées.

- [1] LTRA TECH, « Solutions digitales et développement technologique » et « Services », <https://www.ltratech.com/> ; <https://www.ltratech.com/services>.
- [2] LTRA TECH, « Contact », <https://www.ltratech.com/contact>.
- [3] Microsoft Learn, « What is Azure Virtual Network? », <https://learn.microsoft.com/en-us/azure/virtual-network/virtual-networks-overview>.
- [4] Microsoft Learn, « Azure network security groups overview », <https://learn.microsoft.com/en-us/azure/virtual-network/network-security-groups-overview>.
- [5] Wazuh Documentation, « Architecture », <https://documentation.wazuh.com/current/getting-started/architecture.html>.
- [6] Wazuh Documentation, « Network IDS integration - Suricata », <https://documentation.wazuh.com/current/proof-of-concept-guide/integrate-network-ids-suricata.html>.
- [7] Wazuh Documentation, « localfile » et « JSON decoder », <https://documentation.wazuh.com/current/user-manual/reference/ossec-conf/localfile.html> ; <https://documentation.wazuh.com/current/user-manual/ruleset/decoders/json-decoder.html>.
- [8] Open Information Security Foundation, « Suricata User Guide - EVE JSON », <https://docs.suricata.io/en/suricata-7.0.14/>.
- [9] Canonical, « Auditing an Ubuntu system for CIS Benchmarks compliance », <https://documentation.ubuntu.com/security/compliance/usg/cis-audit/>.
- [10] Canonical, « CIS Benchmarks and profiles », <https://documentation.ubuntu.com/security/compliance/usg/cis-benchmarks/>.
- [11] EMSI, TemplateRapprtStage_4IIR.docx, modèle institutionnel fourni pour la structure du rapport.
- [12] Corpus du projet SOC, captures Azure, Ubuntu, Wazuh, Suricata, Windows et SecureVault, août 2026.
- [13] Wazuh Documentation, « Quickstart installation », <https://documentation.wazuh.com/current/quickstart.html>.
- [14] Wazuh Documentation, « Indexer integration » et « Filebeat », <https://documentation.wazuh.com/current/user-manual/manager/indexer-integration.html>.
- [15] Wazuh Documentation, « Password management », <https://documentation.wazuh.com/current/user-manual/user-administration/password-management.html>.
- [16] Wazuh Documentation, « Installing the Wazuh agent on Windows », <https://documentation.wazuh.com/current/installation-guide/wazuh-agent/wazuh-agent-package-windows.html>.
- [17] Microsoft Learn / Sysinternals, « Sysmon », <https://learn.microsoft.com/en-us/sysinternals/downloads/sysmon>.
- [18] Wazuh Documentation, « Configuring log collection for different operating systems », <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/configuration.html>.
- [19] Wazuh Documentation, « MITRE ATT&CK framework - Sysmon integration », <https://documentation.wazuh.com/current/user-manual/ruleset/mitre.html>.
- [20] Laravel Documentation 12.x, « Logging », <https://laravel.com/framework/docs/12.x/logging>.
- [21] NGINX Documentation, « ngx_http_log_module », https://nginx.org/en/docs/http/ngx_http_log_module.html.
- [22] Wazuh Documentation, « Custom rules », « Rules syntax » et « Testing decoders and rules », <https://documentation.wazuh.com/current/user-manual/ruleset/rules/custom.html> ; <https://documentation.wazuh.com/current/user-manual/ruleset/ruleset-xml-syntax/rules.html> ; <https://documentation.wazuh.com/current/user-manual/ruleset/testing.html>.
- [23] MITRE ATT&CK, « T1110 - Brute Force », <https://attack.mitre.org/techniques/T1110/>.
- [24] MITRE ATT&CK, « T1046 - Network Service Scanning », <https://attack.mitre.org/techniques/T1046/>.
- [25] MITRE ATT&CK, « T1190 - Exploit Public-Facing Application », <https://attack.mitre.org/techniques/T1190/>.
- [26] MITRE ATT&CK, « T1078 - Valid Accounts », <https://attack.mitre.org/techniques/T1078/>.
- [27] Microsoft Learn, « Diagnose a virtual machine network traffic filter problem », <https://learn.microsoft.com/en-us/azure/virtual-network/diagnose-network-traffic-filter-problem>.
- [28] Wazuh Documentation, « Monitoring log files », <https://documentation.wazuh.com/current/user-manual/capabilities/log-data-collection/monitoring-log-files.html>.

Annexes techniques

Annexe A — Commandes essentielles par phase

A.1 Infrastructure et services

```
hostnamectl
ip -br address
df -h /
free -h
sudo systemctl status wazuh-manager --no-pager
sudo systemctl status wazuh-indexer --no-pager
sudo systemctl status wazuh-dashboard --no-pager
sudo systemctl status filebeat --no-pager
```

A.2 Durcissement et CIS

```
sudo ufw status verbose
sudo sshd -t && sudo sshd -T
sudo fail2ban-client status sshd
findmnt /tmp
sudo usg audit cis_level1_server
```

A.3 Suricata et transport

```
sudo suricata -T -c /etc/suricata/suricata.yaml -v
sudo systemctl status suricata --no-pager
sudo grep '2100498' /var/log/suricata/fast.log | tail
sudo tail -n 5 /var/log/suricata/eve.json | jq .
sudo grep -i 'suricata/eve.json' /var/ossec/logs/ossec.log | tail
sudo filebeat test output
```

A.4 Windows et Sysmon

```
Get-Service WazuhSvc, Sysmon64
Get-WinEvent -LogName 'Microsoft-Windows-Sysmon/Operational' -MaxEvents 10
Restart-Service WazuhSvc
# Sur soc01
sudo /var/ossec/bin/agent_control -lc
```

A.5 SecureVault

```
sudo nginx -t
sudo systemctl status nginx php*-fpm mysql wazuh-agent --no-pager
cd /var/www/securevault/backend
php artisan about
php artisan route:list
sudo tail -n 10 storage/logs/security.log
sudo /var/ossec/bin/wazuh-logtest
```

Annexe B — Extraits de configuration

B.1 Suricata vers Wazuh

```
# /etc/suricata/suricata.yaml
vars:
  address-groups:
    HOME_NET: "[192.168.0.0/16,10.0.0.0/8,172.16.0.0/12]"
af-packet:
  - interface: eth0

# /var/ossec/etc/ossec.conf
```

```
<localfile>
<location>/var/log/suricata/eve.json</location>
<log_format>json</log_format>
</localfile>
```

B.2 Windows EventChannel

```
<localfile>
<location>Microsoft-Windows-Sysmon/Operational</location>
<log_format>eventchannel</log_format>
</localfile>
```

B.3 Sources web01

```
<localfile>
<location>/var/www/securevault/backend/storage/logs/security.log</location>
<log_format>json</log_format>
<label key="app.name">securevault</label>
</localfile>
<localfile>
<location>/var/log/nginx/access.log</location>
<log_format>apache</log_format>
</localfile>
```

Annexe C — Matrice synthétique des preuves

Tableau 17 : Matrice synthétique des résultats techniques

Phase	Objet	Preuve	Conclusion
1	soc01 et Wazuh	services systemd et accès Dashboard	socle central opérationnel
1	stockage	df et du avant/après	saturation identifiée et corrigée
2	durcissement	NSG, UFW, sshd, Fail2ban et findmnt	contrôles appliqués par couches
2	CIS	rapports USG	106 puis 5 écarts
3	Suricata	SID 2100498 et eve.json	détection réseau produite
3	transport	filebeat test output	connexion TLS vers l'Indexer
4	SIEM	filtres et dashboards sauvegardés	investigation structurée
5	win01	agent 001 et événement Sysmon	source Windows indexée
6	SecureVault	security.log, Nginx et agent web01	source applicative centralisée

Annexe D — Protection des informations sensibles

Les preuves ont été recadrées ou masquées lorsque leur contenu pouvait faciliter un accès non autorisé. Les données nécessaires à l'analyse, telles que les adresses privées ou les noms d'agents, sont conservées uniquement lorsqu'elles expliquent l'architecture.

- Masquer les adresses IP publiques et les sources personnelles.
- Ne jamais afficher les mots de passe, clés SSH, jetons, cookies ou codes d'activation.
- Éviter le contenu des documents SecureVault et les données personnelles des comptes.
- Conserver la commande, le résultat utile, l'horodatage et le contexte technique.